

DEEP LEARNING FUNDAMENTALS

A Conceptual Introduction

Personal Study Book

Md. Shaon Khan

Institute of Information Technology
Jahangirnagar University
2026

সূচিপত্র (Table of Contents)

Contents

Preface	7
How to Use This Book	8
AI, ML ও Deep Learning-এর ভিত্তি	9
Artificial Intelligence, Machine Learning এবং Deep Learning	10
১.১ Artificial Intelligence	10
১.২ Machine Learning	10
১.৩ Deep Learning	10
১.৪ AI → ML → DL সম্পর্ক	11
১.৫ Machine Learning বনাম Deep Learning	11
Deep Learning কেন শক্তিশালী হয়ে উঠল	15
Feature Engineering ও Automatic Feature Extraction	16
২.১ Feature	16
২.২ Feature Engineering	16
২.৩ Traditional ML-এ Feature Engineering	16
২.৪ Automatic Feature Extraction	16
২.৫ Representation Learning	16
২.৬ Feature Engineering বনাম Deep Learning	16
Unstructured Data, Data Hungry এবং Scalability	19
৩.১ Structured Data	19
৩.২ Unstructured Data	19
৩.৩ ছবি, টেক্সট, অডিও, ভিডিও ও Signal কঠিন কেন	19
৩.৪ Deep Learning কেন Unstructured Data-র জন্য উপযোগী	19
৩.৫ Deep Learning কেন Data Hungry	19
৩.৬ বৃহৎ Dataset ও Generalization	19
৩.৭ Data Quality বনাম Data Quantity	20
৩.৮ High Scalability	20
জৈবিক নিউরন থেকে কৃত্রিম নিউরন	22
Biological Neuron	23
৪.১ জৈবিক নিউরনের গঠন	23
৪.২ জৈবিক নিউরন থেকে কৃত্রিম নিউরন	23
Artificial Neuron	26
৫.১ Artificial Neuron-এর গঠন	26
৫.২ প্রতিটি উপাদানের ব্যাখ্যা	26

Perceptron	28
Perceptron	29
৬.১ Perceptron কী?	29
৬.২–৬.৫ Input, Weight, Bias এবং Activation	29
৬.৬ Binary Classification	29
Weight বিস্তারিত	29
Bias বিস্তারিত	30
Single-Layer Perceptron এবং Linear Separability	31
৭.১–৭.৩ Single-Layer Perceptron, Decision Boundary ও Linear Separability	31
৭.৪–৭.৬ AND, OR এবং XOR	31
কেন AND ও OR পৃথকযোগ্য, কিন্তু XOR নয়	31
মাল্টি-লেয়ার নিউরাল নেটওয়ার্ক	33
কেন একাধিক Layer প্রয়োজন	34
৮.১ সীমাবদ্ধতা থেকে সমাধান	34
Input Layer, Hidden Layers এবং Output Layer	36
Input Layer	36
Hidden Layer	36
Output Layer	36
Layer-ভিত্তিক প্যাটার্ন শেখা	36
Deep Learning কীভাবে শেখে	40
Learnable Parameters	41
Parameter কী?	41
কেন 'Learnable'	41
Forward Propagation	43
Forward Propagation-এর প্রবাহ	43
Loss Function	46
কেন Loss প্রয়োজন	46
সাধারণ Loss Function	46
Backpropagation	48
১৩.১ Backpropagation কী?	48
১৩.২ কেন Backpropagation প্রয়োজন	48
১৩.৩–১৩.৪ Error Contribution ও Gradient	48
১৩.৫ Chain Rule ধারণা	48
১৩.৬–১৩.৭ Weight ও Bias Gradient	49
Optimization	50
১৪.১ Optimizer	50

১৪.২ Gradient Descent	50
১৪.৩ Learning Rate	50
১৪.৪ Parameter Update	50
সম্পূর্ণ Training Cycle	50
হার্ডওয়্যার কেন গুরুত্বপূর্ণ	53
CPU এবং GPU	54
১৫.১ CPU	54
১৫.২–১৫.৩ GPU ও Graphics Card	54
১৫.৪ Parallelism	54
১৫.৫ Deep Learning কেন Parallelism থেকে উপকৃত হয়	54
NVIDIA, CUDA এবং GPU Computing	56
১৬.১–১৬.২ NVIDIA ও CUDA	56
১৬.৩ CUDA বনাম GPU	56
১৬.৪ cuDNN	56
১৬.৫ কেন NVIDIA প্রভাবশালী হয়ে উঠল	56
১৬.৬–১৬.৭ AMD ও ROCm	56
RAPIDS	58
RAPIDS কী?	58
স্পষ্টীকরণ	58
কেন ২০১২ একটি টার্নিং পয়েন্ট ছিল	59
Deep Learning বিপ্লবের পূর্বে	60
১৮.১ পর্যাপ্ত Large Labeled Data-র অভাব	60
১৮.২ সীমিত Compute Power	60
১৮.৩ পূর্ববর্তী Training/Architecture সীমাবদ্ধতা	60
বাধা দূর হওয়া	60
Fei-Fei Li এবং ImageNet	62
ImageNet ও Fei-Fei Li	62
Data Annotation / Data Labeling	62
ILSVRC	64
ILSVRC কী?	64
২০১২ ILSVRC এবং AlexNet	65
২১.১–২১.৪ AlexNet ও এর নির্মাতারা	65
২১.৫–২১.৬ Deep CNN ও GPU Acceleration	65
২১.৭–২১.৯ ImageNet, ২০১২ ফলাফল ও ঐতিহাসিক গুরুত্ব	65
নিউরাল নেটওয়ার্ক আর্কিটেকচারের বিবর্তন	68
ANN / MLP	69

সাধারণ-উদ্দেশ্য Architecture	69
CNN	70
CNN কেন তৈরি হয়েছিল	70
Convolution, Kernel/Filter ও Feature Map	70
Hierarchical Feature Learning	70
RNN, LSTM এবং GRU	73
RNN ও Sequential Data	73
LSTM ও GRU	73
Attention এবং Transformer	75
২৫.১–২৫.২ কেন Attention প্রয়োজন	75
২৫.৩ Self-Attention	75
২৫.৪–২৫.৫ Transformer ও এর গুরুত্ব	75
LLM এবং আধুনিক Generative AI	77
Large Language Model	78
LLM কী?	78
Token, Tokenization ও Context	78
Training ও Inference	78
Next-token Prediction ও Generation Process	78
ChatGPT এবং Gemini	82
ChatGPT ও Gemini কী	82
Product/System বনাম Underlying Model	82
Multimodal ক্ষমতা	82
সম্পূর্ণ সংযোগ	84
নিউরন থেকে আধুনিক AI পর্যন্ত	85
সম্পূর্ণ যাত্রা	85
শেখার প্রক্রিয়ার সংযোগ	85
Historical Timeline	88
Final Summary	90
Glossary	91
Quick Revision Sheet	95
Core Definitions	95
Core Equations	95
Core Architecture Diagram	96
Historical Timeline (সংক্ষেপে)	96
AI → ML → DL সম্পর্ক	96
Training Pipeline	97

ANN → CNN → RNN → Transformer → LLM.....	97
CPU → GPU → CUDA	97
ImageNet → ILSVRC → AlexNet.....	97
Final Concept Map.....	99
References.....	100

Preface

এই বইটি তৈরি করা হয়েছে Deep Learning-এর মৌলিক ধারণাগুলো একটি স্বয়ংসম্পূর্ণ, ধারাবাহিক ও পুনরায় পাঠযোগ্য রূপে সংকলিত করার উদ্দেশ্যে। বর্তমান সময়ে Deep Learning একটি অত্যন্ত দ্রুত বিকাশমান ক্ষেত্র, এবং এর সাথে সম্পর্কিত তথ্য বিভিন্ন উৎসে ছড়িয়ে-ছিটিয়ে থাকে। এই বইয়ের লক্ষ্য হলো সেই বিক্ষিপ্ত ধারণাগুলোকে একটি যৌক্তিক ক্রমে সাজিয়ে একটি ব্যক্তিগত অধ্যয়ন-গ্রন্থ (personal study book) হিসেবে উপস্থাপন করা।

যেকোনো জটিল বিষয় গভীরভাবে বোঝার পূর্বশর্ত হলো তার ভিত্তি সুদৃঢ়ভাবে আয়ত্ত করা। CNN, RNN, Transformer বা LLM-এর মতো আধুনিক architecture বোঝার আগে Artificial Neuron, Perceptron, Forward Propagation, Backpropagation-এর মতো মৌলিক ধারণা স্পষ্টভাবে বোঝা প্রয়োজন। এই বইটি ঠিক সেই ভিত্তি তৈরির উদ্দেশ্যেই রচিত।

এটি একটি ভূমিকামূলক (introductory) গ্রন্থ। এখানে উন্নত গণিত, গবেষণা-স্তরের বিস্তারিত আলোচনা, বা architecture-এর অভ্যন্তরীণ বাস্তবায়ন নিয়ে গভীরভাবে আলোচনা করা হয়নি। বরং প্রতিটি ধারণা ধারণাগতভাবে (conceptually) স্পষ্ট করার চেষ্টা করা হয়েছে, যাতে পাঠক একটি শক্তিশালী conceptual foundation তৈরি করতে পারেন এবং প্রয়োজনে পরবর্তীতে আরও গভীর গবেষণাধর্মী উপকরণে অগ্রসর হতে পারেন।

How to Use This Book

এই বইটি ক্রমানুসারে পড়ার জন্য ডিজাইন করা হয়েছে—প্রতিটি অধ্যায় পরবর্তী অধ্যায়ের ভিত্তি তৈরি করে। প্রতিটি অধ্যায়ে Learning Objectives দিয়ে শুরু করে মূল ধারণা (Main Concept), গুরুত্বপূর্ণ পরিভাষা (Key Terms), প্রয়োজনে উদাহরণ ও চিত্র, এবং শেষে Key Takeaways ও Quick Review Questions দেওয়া হয়েছে।

প্রথমবার পড়ার সময় প্রতিটি অধ্যায়ের Learning Objectives পড়ে নিন, তারপর মূল বিষয়বস্তু পড়ুন এবং Quick Review Questions-এর মাধ্যমে নিজের বোঝাপড়া যাচাই করুন। ভবিষ্যতে দ্রুত পুনরালোচনার জন্য বইয়ের শেষে থাকা Glossary এবং Quick Revision Sheet ব্যবহার করুন।

যেকোনো ইংরেজি পরিভাষা প্রথমবার ব্যবহারের সময় তার সংক্ষিপ্ত বাংলা ব্যাখ্যা দেওয়া হয়েছে। এরপর সেই পরিভাষাটি সামঞ্জস্যপূর্ণভাবে ইংরেজিতেই ব্যবহার করা হয়েছে, যাতে আন্তর্জাতিক প্রযুক্তিগত সাহিত্যের সাথে সামঞ্জস্য বজায় থাকে।

PART I

AI, ML ও Deep Learning-এর ভিত্তি

Chapter 1

Artificial Intelligence, Machine Learning এবং Deep Learning

Deep Learning

Deep Learning হলো Machine Learning-এর একটি শাখা যেখানে বহুস্তরবিশিষ্ট Neural Network ব্যবহার করে data থেকে complex patterns এবং representations শেখানো হয়।

Learning Objectives

- Artificial Intelligence, Machine Learning এবং Deep Learning-এর সংজ্ঞা ব্যাখ্যা করতে পারা
- AI, ML ও DL-এর মধ্যে ক্রমসম্পর্ক (hierarchy) বোঝা
- Machine Learning ও Deep Learning-এর মধ্যে মূল পার্থক্য চিহ্নিত করতে পারা
- Deep Learning কেন এবং কখন বেশি কার্যকর তা ব্যাখ্যা করতে পারা

মূল ধারণা (Main Concept)

১.১ Artificial Intelligence

Artificial Intelligence (AI) বলতে বোঝায় কম্পিউটার সিস্টেমকে এমনভাবে তৈরি করা, যাতে সেটি মানুষের বুদ্ধিবৃত্তিক আচরণের মতো কাজ করতে পারে—যেমন সিদ্ধান্ত নেওয়া, সমস্যার সমাধান করা, ভাষা বোঝা বা প্যাটার্ন চেনা। AI একটি বিস্তৃত ক্ষেত্র, যার মধ্যে rule-based system থেকে শুরু করে আধুনিক Machine Learning পর্যন্ত অনেক ধরনের পদ্ধতি অন্তর্ভুক্ত।

AI-এর সাধারণ উদাহরণের মধ্যে রয়েছে গেম খেলার প্রোগ্রাম, স্বয়ংক্রিয় সুপারিশ ব্যবস্থা (recommendation system), কণ্ঠস্বর সহকারী (voice assistant), এবং স্বচালিত গাড়ি (self-driving car)। এই সব ক্ষেত্রেই মূল লক্ষ্য একটাই—নির্দিষ্ট একটি কাজ বুদ্ধিমত্তার সাথে সম্পন্ন করা।

১.২ Machine Learning

Machine Learning (ML) হলো AI-এর একটি উপশাখা, যেখানে সিস্টেমকে প্রতিটি নিয়ম আলাদাভাবে প্রোগ্রাম না করে data থেকে patterns শিখতে দেওয়া হয়। অর্থাৎ, মানুষ প্রতিটি সিদ্ধান্তের নিয়ম হাতে লিখে দেয় না; বরং model নিজেই training data বিশ্লেষণ করে সিদ্ধান্ত নেওয়ার পদ্ধতি শেখে।

একটি ML সিস্টেমের কাজ সাধারণত দুই ধাপে বিভক্ত: **Training** (data থেকে শেখা) এবং **Prediction** (নতুন data-এর উপর সিদ্ধান্ত প্রয়োগ করা)। Training ধাপে model একটি dataset বিশ্লেষণ করে input ও output-এর মধ্যে সম্পর্ক অনুমান করে, এবং Prediction ধাপে সেই শেখা সম্পর্ক ব্যবহার করে নতুন, আগে না-দেখা data সম্পর্কে সিদ্ধান্ত দেয়।

১.৩ Deep Learning

Deep Learning (DL) হলো Machine Learning-এরই একটি বিশেষ শাখা, যেখানে বহুস্তরবিশিষ্ট (multi-layer) **Neural Network** ব্যবহার করে data থেকে জটিল patterns এবং representations শেখানো হয়। এখানে 'Deep' শব্দটি ব্যবহৃত হয় কারণ network-এ একাধিক hidden layer থাকে—layer-এর সংখ্যা যত বেশি, network তত 'গভীর' (deep) বলে বিবেচিত হয়।

প্রতিটি স্তর (layer) input data-এর ভিন্ন ভিন্ন স্তরের বৈশিষ্ট্য শেখে—শুরুর স্তরগুলো সাধারণ প্যাটার্ন এবং পরের স্তরগুলো ক্রমশ জটিল ও বিমূর্ত (abstract) প্যাটার্ন শেখে। এই স্তরভিত্তিক শেখার কারণেই Deep Learning ছবি, ভাষা, শব্দের মতো জটিল ও unstructured data-এ কার্যকর।

১.৪ AI → ML → DL সম্পর্ক

AI, ML এবং DL একে অপরের সাথে একটি ধারাবাহিক ক্রমসম্পর্কে (hierarchy) যুক্ত। Machine Learning হলো Artificial Intelligence-এর একটি উপশাখা, এবং Deep Learning হলো Machine Learning-এর একটি উপশাখা। অর্থাৎ, প্রতিটি Deep Learning পদ্ধতি একই সাথে Machine Learning এবং AI-ও বটে, কিন্তু প্রতিটি ML পদ্ধতি Deep Learning নয়।

১.৫ Machine Learning বনাম Deep Learning

Machine Learning এবং Deep Learning উভয়ই data থেকে শেখে, কিন্তু তাদের পদ্ধতিতে গুরুত্বপূর্ণ পার্থক্য রয়েছে। ঐতিহ্যগত (traditional) ML পদ্ধতিতে সাধারণত মানুষকে হাতে **Feature Engineering** করতে হয়—অর্থাৎ, কোন বৈশিষ্ট্যগুলো গুরুত্বপূর্ণ তা নির্ধারণ করে দিতে হয়। অন্যদিকে, Deep Learning model অনেকাংশে নিজে থেকেই raw data থেকে useful feature শিখে নিতে পারে।

এখানে একটি গুরুত্বপূর্ণ বিষয় স্পষ্ট করা প্রয়োজন: Deep Learning সবসময় traditional ML-এর চেয়ে ভালো ফল দেয় না। **Structured/Tabular Data** এবং তুলনামূলক ছোট dataset-এর ক্ষেত্রে traditional ML প্রায়ই দ্রুততর এবং কার্যকর হয়। Deep Learning মূলত বড় আকারের এবং জটিল **Unstructured Data**-এর ক্ষেত্রে বিশেষভাবে শক্তিশালী।

Key Terms

Artificial Intelligence • Machine Learning • Deep Learning • Neural Network • Training • Prediction • Structured Data • Unstructured Data

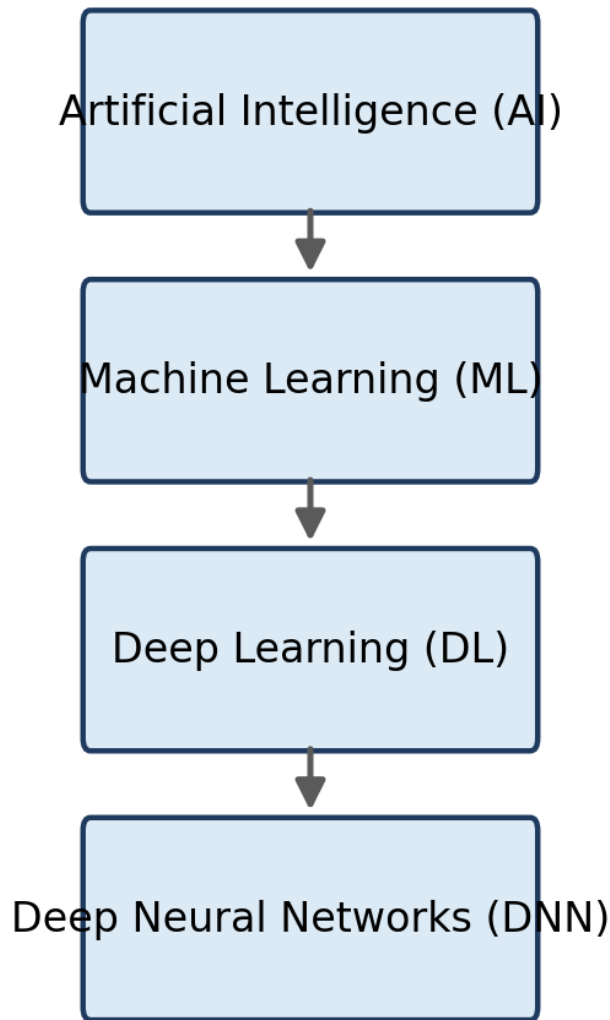
সারণি ১.১: AI বনাম ML বনাম DL

বিষয়	Artificial Intelligence	Machine Learning	Deep Learning
সংজ্ঞা	বুদ্ধিমান আচরণ অনুকরণকারী সিস্টেম	Data থেকে শেখা পদ্ধতি	বহুস্তর Neural Network-ভিত্তিক পদ্ধতি
সম্পর্ক	সবচেয়ে বিস্তৃত ক্ষেত্র	AI-এর উপশাখা	ML-এর উপশাখা

বিষয়	Artificial Intelligence	Machine Learning	Deep Learning
উদাহরণ	Rule-based system, robotics	Decision Tree, SVM	CNN, RNN, Transformer

সারণি ১.২: Traditional ML বনাম Deep Learning

বৈশিষ্ট্য	Traditional ML	Deep Learning
Feature Engineering	সাধারণত হাতে করতে হয়	অনেকাংশে স্বয়ংক্রিয়
উপযুক্ত Data	Structured / Tabular Data	Unstructured Data (image, text, audio)
Data-এর প্রয়োজন	তুলনামূলক কম data-তেও কার্যকর	সাধারণত বেশি data প্রয়োজন
Computation-এর প্রয়োজন	তুলনামূলক কম	সাধারণত বেশি (GPU-নির্ভর)
Scalability	বড় data-তে সীমিত উন্নতি	বড় data ও model-এ ভালো scale করে



চিত্র ১.১: Artificial Intelligence → Machine Learning → Deep Learning → Deep Neural Networks ক্রমসম্পর্ক

Key Takeaways

- AI সবচেয়ে বিস্তৃত ক্ষেত্র; ML তার উপশাখা এবং DL হলো ML-এর একটি বিশেষ শাখা।
- Machine Learning মানুষের হাতে-লেখা নিয়মের পরিবর্তে data থেকে patterns শেখে।
- Deep Learning বহুস্তর Neural Network ব্যবহার করে, যেখানে 'Deep' মানে একাধিক hidden layer।
- Traditional ML-এ সাধারণত হাতে Feature Engineering প্রয়োজন হয়, Deep Learning-এ তা অনেকাংশে স্বয়ংক্রিয়।
- Deep Learning সবসময় সেরা সমাধান নয়—structured ও ছোট dataset-এ traditional ML প্রায়ই বেশি কার্যকর।

Quick Review Questions

1. Artificial Intelligence, Machine Learning এবং Deep Learning-এর মধ্যে সম্পর্ক ব্যাখ্যা করুন।
2. Deep Learning-কে কেন 'Deep' বলা হয়?

3. Traditional ML এবং Deep Learning-এর মধ্যে Feature Engineering-এর দিক থেকে পার্থক্য কী?
4. কোন পরিস্থিতিতে traditional ML, Deep Learning-এর চেয়ে বেশি উপযোগী হতে পারে?

PART II

Deep Learning কেন শক্তিশালী হয়ে উঠল

Chapter 2

Feature Engineering ও Automatic Feature Extraction

Learning Objectives

- Feature এবং Feature Engineering-এর অর্থ ব্যাখ্যা করতে পারা
- Traditional ML-এ Feature Engineering-এর ভূমিকা বোঝা
- Automatic Feature Extraction এবং Representation Learning-এর ধারণা বোঝা

মূল ধারণা (Main Concept)

২.১ Feature

Feature হলো data-এর এমন একটি পরিমাপযোগ্য বৈশিষ্ট্য, যা model-কে সিদ্ধান্ত নিতে সাহায্য করে। যেমন একটি বাড়ির দাম অনুমান করার model-এ বাড়ির আয়তন, রুমের সংখ্যা, বা অবস্থান—এগুলো সবই feature।

২.২ Feature Engineering

Feature Engineering বলতে বোঝায় raw data থেকে মানুষের জ্ঞান ও domain understanding ব্যবহার করে model-এর জন্য উপযোগী feature তৈরি বা নির্বাচন করার প্রক্রিয়া। এটি একটি সময়সাপেক্ষ কাজ, কারণ কোন feature গুরুত্বপূর্ণ তা নির্ধারণ করতে গভীর domain জ্ঞান প্রয়োজন হয়।

২.৩ Traditional ML-এ Feature Engineering

Traditional ML model, যেমন Decision Tree বা Linear Regression, সাধারণত raw data (যেমন একটি ছবির প্রতিটি pixel) সরাসরি বুঝতে পারে না। তাই মানুষকে হাতে feature তৈরি করে দিতে হয়—যেমন ছবির ক্ষেত্রে edge, color histogram, বা texture-এর মতো পরিসংখ্যান বের করা।

২.৪ Automatic Feature Extraction

Deep Learning model, বিশেষ করে বহুস্তর Neural Network, training-এর মাধ্যমে নিজে থেকেই raw data থেকে useful feature চিনতে শেখে। একে **Automatic Feature Extraction** বলা হয়। এর ফলে হাতে feature তৈরি করার প্রয়োজন অনেকাংশে কমে যায়, যদিও সম্পূর্ণরূপে দূর হয় না।

২.৫ Representation Learning

Representation Learning হলো একটি বিস্তৃত ধারণা, যেখানে model data-কে এমনভাবে রূপান্তরিত (represent) করতে শেখে যা পরবর্তী কাজের (যেমন classification) জন্য সবচেয়ে উপযোগী। Automatic Feature Extraction মূলত Representation Learning-এরই একটি ফলাফল।

২.৬ Feature Engineering বনাম Deep Learning

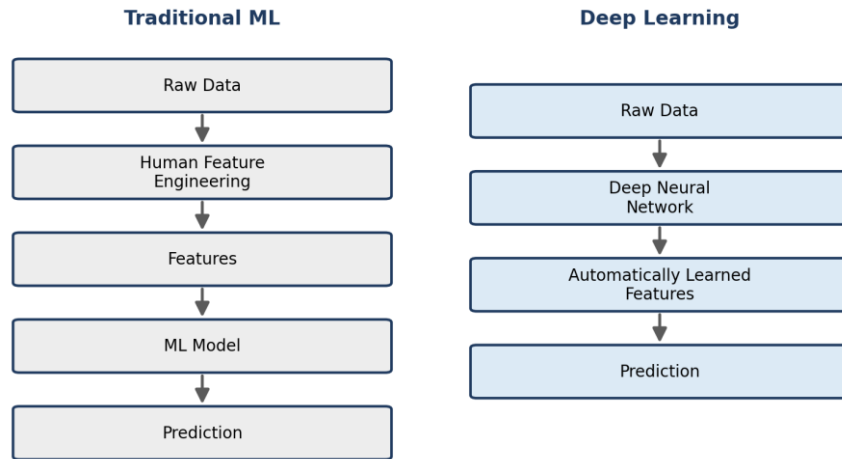
নিচের প্রবাহচিত্র দুটি পদ্ধতির মধ্যে পার্থক্য স্পষ্ট করে। এখানে গুরুত্বপূর্ণ বিষয় হলো—Deep Learning সব ধরনের data preparation বা domain জ্ঞানের প্রয়োজনীয়তা সম্পূর্ণরূপে দূর করে না। Data পরিষ্কার করা (cleaning), সঠিকভাবে label করা, এবং উপযুক্ত architecture নির্বাচন করার জন্য এখনো মানুষের সিদ্ধান্ত ও domain জ্ঞান প্রয়োজন হয়।

Key Terms

Feature • Feature Engineering • Feature Extraction • Automatic Feature Extraction • Representation Learning

সারণি ২.১: Feature Engineering বনাম Automatic Feature Extraction

বৈশিষ্ট্য	Feature Engineering (Traditional ML)	Automatic Feature Extraction (DL)
কে feature তৈরি করে	মানুষ (domain expert)	Neural Network নিজে (training-এর মাধ্যমে)
সময় ও শ্রম	তুলনামূলক বেশি	প্রাথমিক পর্যায়ে কম, কিন্তু training-এ বেশি compute প্রয়োজন
উপযুক্ততা	Structured/Tabular data	Image, Text, Audio-এর মতো Unstructured data



চিত্র ২.১: Traditional ML বনাম Deep Learning workflow-এর তুলনা

Key Takeaways

- Feature হলো data-এর পরিমাপযোগ্য বৈশিষ্ট্য যা model সিদ্ধান্তে ব্যবহার করে।
- Traditional ML-এ সাধারণত হাতে Feature Engineering প্রয়োজন হয়।
- Deep Learning model training-এর মাধ্যমে নিজে থেকেই feature শিখতে পারে—একে Automatic Feature Extraction বলা হয়।

- Representation Learning হলো data-কে উপযোগীভাবে রূপান্তরিত করে শেখার বিস্তৃত ধারণা।
- Deep Learning সম্পূর্ণরূপে মানুষের সম্পৃক্ততা দূর করে না—data preparation ও domain জ্ঞান এখনও প্রয়োজন।

Quick Review Questions

5. Feature Engineering বলতে কী বোঝায়?
6. Automatic Feature Extraction কীভাবে Feature Engineering-এর প্রয়োজনীয়তা কমায়?
7. Representation Learning-এর সাথে Automatic Feature Extraction-এর সম্পর্ক কী?
8. Deep Learning কি সম্পূর্ণভাবে মানুষের সম্পৃক্ততা দূর করে? ব্যাখ্যা করুন।

Chapter 3

Unstructured Data, Data Hungry এবং Scalability

Learning Objectives

- Structured ও Unstructured Data-এর পার্থক্য বোঝা
- Deep Learning কেন 'Data Hungry' তা ব্যাখ্যা করতে পারা
- Data ও Compute বৃদ্ধির সাথে performance-এর সম্পর্ক বোঝা

মূল ধারণা (Main Concept)

৩.১ Structured Data

Structured Data হলো এমন data যা row ও column-এর মতো একটি সুনির্দিষ্ট কাঠামোতে সংরক্ষিত থাকে—যেমন একটি spreadsheet বা database table। প্রতিটি column-এর অর্থ স্পষ্ট এবং সহজে সংজ্ঞায়িত।

৩.২ Unstructured Data

Unstructured Data হলো এমন data যার কোনো নির্দিষ্ট সারি-কলাম কাঠামো নেই—যেমন ছবি, টেক্সট, অডিও, ভিডিও, বা সেন্সর signal। এই ধরনের data-তে অর্থপূর্ণ প্যাটার্ন খুঁজে বের করা তুলনামূলক কঠিন।

৩.৩ ছবি, টেক্সট, অডিও, ভিডিও ও Signal কঠিন কেন

এই ধরনের data-তে গুরুত্বপূর্ণ তথ্য প্রায়ই জটিল, স্তরভিত্তিক (hierarchical), এবং প্রেক্ষাপট-নির্ভর (context-dependent) সম্পর্কের মধ্যে লুকিয়ে থাকে। যেমন একটি ছবিতে বস্তু চেনার জন্য শুধু আলাদা pixel নয়, বরং pixel-গুলোর মধ্যকার স্থানিক (spatial) সম্পর্ক বোঝা প্রয়োজন।

৩.৪ Deep Learning কেন Unstructured Data-র জন্য উপযোগী

বহুস্তর Neural Network ধাপে ধাপে সরল থেকে জটিল প্যাটার্ন শিখতে পারে, যা unstructured data-র জটিল কাঠামো বোঝার জন্য বিশেষভাবে উপযোগী। এই কারণে Computer Vision, Natural Language Processing এবং Speech Recognition-এর মতো ক্ষেত্রে Deep Learning ব্যাপকভাবে ব্যবহৃত হয়।

৩.৫ Deep Learning কেন Data Hungry

Deep Learning model-এ সাধারণত অনেক বেশি Learnable Parameters থাকে, যেগুলোর সঠিক মান নির্ধারণের জন্য প্রচুর পরিমাণ training data প্রয়োজন হয়। পর্যাপ্ত data ছাড়া model সহজেই training data মুখস্থ করে ফেলতে পারে, যা Overfitting-এর দিকে নিয়ে যায়।

৩.৬ বৃহৎ Dataset ও Generalization

Generalization বলতে বোঝায় model-এর ক্ষমতা—নতুন, আগে না-দেখা data-তেও সঠিকভাবে কাজ করা। বৃহৎ ও বৈচিত্র্যময় dataset সাধারণত ভালো generalization নিশ্চিত করতে সাহায্য করে।

৩.৭ Data Quality বনাম Data Quantity

শুধু data-র পরিমাণ যথেষ্ট নয়; data-র গুণগত মান (quality)—যেমন সঠিক label, কম noise, এবং বৈচিত্র্য—সমানভাবে গুরুত্বপূর্ণ। নিম্নমানের বিশাল dataset প্রায়ই উচ্চমানের ছোট dataset-এর চেয়ে খারাপ ফল দিতে পারে।

৩.৮ High Scalability

Deep Learning-এর একটি গুরুত্বপূর্ণ বৈশিষ্ট্য হলো এর **Scalability**—অর্থাৎ, data, model-এর আকার এবং computation বাড়ানোর সাথে সাথে performance বৃদ্ধি পাওয়ার প্রবণতা। এটি একটি সাধারণ প্রবণতা, কোনো নিরঙ্কুশ নিয়ম নয়।

Traditional ML model প্রায়ই একটি নির্দিষ্ট পরিমাণ data-র পর performance-এ diminishing returns (ক্রমহ্রাসমান উন্নতি) দেখায়, অর্থাৎ আরও data যোগ করলেও performance খুব বেশি বাড়ে না। কিন্তু পর্যাপ্ত বড় Deep Learning model প্রায়শই আরও বেশি data ও computation থেকে উপকৃত হতে থাকে।

Key Terms

Structured Data • Unstructured Data • Data Hungry • Generalization • Overfitting • Scalability

সারণি ৩.১: Structured বনাম Unstructured Data

বৈশিষ্ট্য	Structured Data	Unstructured Data
কাঠামো	সারি-কলাম ভিত্তিক	নির্দিষ্ট কাঠামো নেই
উদাহরণ	Spreadsheet, Database Table	Image, Text, Audio, Video
সাধারণ উপযোগী পদ্ধতি	Traditional ML	Deep Learning

Key Takeaways

- Structured data একটি নির্দিষ্ট কাঠামোতে থাকে; unstructured data-র তা থাকে না।
- Deep Learning model-এর অনেক Learnable Parameters থাকায় এটি সাধারণত বেশি training data দাবি করে।
- Data Quality ও Data Quantity উভয়ই গুরুত্বপূর্ণ—শুধু বেশি data যথেষ্ট নয়।
- Deep Learning সাধারণত data ও compute বাড়ানোর সাথে ভালো scale করে, যেখানে traditional ML-এ diminishing returns দেখা যেতে পারে।

Quick Review Questions

9. Structured ও Unstructured Data-র মধ্যে পার্থক্য কী?
10. Deep Learning-কে কেন 'Data Hungry' বলা হয়?
11. Data Quality এবং Data Quantity-র মধ্যে কোনটি বেশি গুরুত্বপূর্ণ হতে পারে এবং কেন?
12. Scalability বলতে Deep Learning-এর প্রেক্ষাপটে কী বোঝায়?

PART III

জৈবিক নিউরন থেকে কৃত্রিম নিউরন

Chapter 4

Biological Neuron

Learning Objectives

- মানুষের মস্তিষ্কের নিউরনের মূল অংশগুলো চিহ্নিত করতে পারা
- নিউরনের মধ্যে তথ্য প্রবাহের ধারণা বোঝা
- Artificial Neural Network কীভাবে জৈবিক নিউরন দ্বারা অনুপ্রাণিত তা ব্যাখ্যা করা

মূল ধারণা (Main Concept)

৪.১ জৈবিক নিউরনের গঠন

মানুষের মস্তিষ্কে কোটি কোটি **Neuron** রয়েছে, যেগুলো একসাথে কাজ করে তথ্য প্রক্রিয়াকরণ করে। প্রতিটি জৈবিক নিউরনের কয়েকটি মূল অংশ থাকে: **Dendrites** (অন্যান্য নিউরন থেকে সংকেত গ্রহণ করে), **Cell Body / Nucleus** (গ্রহীত সংকেত প্রক্রিয়াকরণ করে), **Axon** (প্রক্রিয়াকৃত সংকেত পরিবহন করে), এবং **Axon Terminal** (সংকেতটি পরবর্তী নিউরনে পাঠায়)।

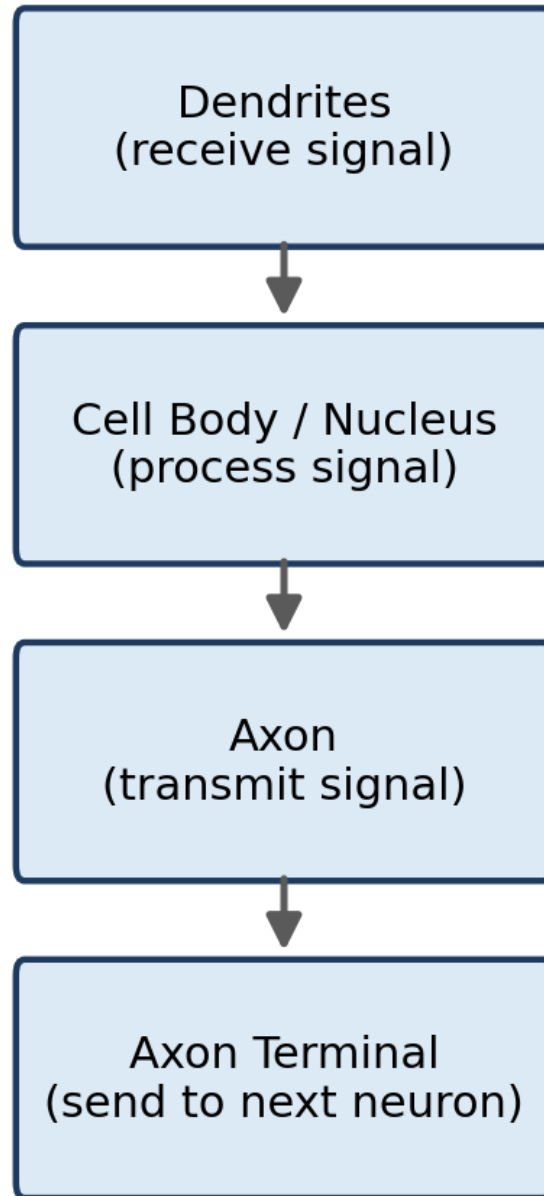
তথ্য প্রবাহের ক্রম নিচের চিত্রে দেখানো হয়েছে: Dendrites সংকেত গ্রহণ করে, Cell Body তা প্রক্রিয়া করে, Axon-এর মাধ্যমে তা পরিবাহিত হয়, এবং Axon Terminal থেকে তা পরবর্তী নিউরনের Dendrites-এ পৌঁছায়।

৪.২ জৈবিক নিউরন থেকে কৃত্রিম নিউরন

Artificial Neural Network জৈবিক মস্তিষ্কের নিউরন দ্বারা অনুপ্রাণিত হয়ে তৈরি করা হয়েছে, তবে এটি স্পষ্টভাবে উল্লেখ করা প্রয়োজন যে Artificial Neural Network একটি গাণিতিক ও কম্পিউটেশনাল মডেল—এটি জৈবিক মস্তিষ্কের হুবহু প্রতিক্রিয়া নয়। কৃত্রিম নিউরনে Dendrites-এর ভূমিকা পালন করে **Input**, Cell Body-এর ভূমিকা পালন করে **Weighted Sum & Activation Function**, এবং Axon-এর ভূমিকা পালন করে **Output**।

Key Terms

Neuron • Dendrites • Cell Body • Nucleus • Axon • Axon Terminal • Artificial Neural Network



চিত্র ৪.১: জৈবিক নিউরনে তথ্য প্রবাহ — *Dendrites → Cell Body → Axon → Axon Terminal*

Key Takeaways

- জৈবিক নিউরনের মূল অংশ: Dendrites, Cell Body, Nucleus, Axon এবং Axon Terminal।
- তথ্য Dendrites দিয়ে প্রবেশ করে, Cell Body-তে প্রক্রিয়া হয়, এবং Axon-এর মাধ্যমে বের হয়ে যায়।
- Artificial Neural Network জৈবিক নিউরন দ্বারা অনুপ্রাণিত, কিন্তু এটি একটি গাণিতিক মডেল, জৈবিক মস্তিষ্কের হুবহু প্রতিক্রিয়া নয়।

Quick Review Questions

13. জৈবিক নিউরনের মূল অংশগুলো কী কী?
14. জৈবিক নিউরনে তথ্য প্রবাহের ক্রম ব্যাখ্যা করুন।

15. Artificial Neural Network জৈবিক মস্তিষ্কের হুবহু প্রতিক্রিয়া নয় কেন?

Chapter 5

Artificial Neuron

Neuron

Neuron হলো Neural Network-এর মৌলিক একক, যা একাধিক input গ্রহণ করে, Weight ও Bias ব্যবহার করে একটি Weighted Sum গণনা করে, এবং Activation Function প্রয়োগ করে চূড়ান্ত output তৈরি করে।

Learning Objectives

- Artificial Neuron-এর গাণিতিক গঠন ব্যাখ্যা করতে পারা
- Weight, Bias এবং Activation Function-এর ভূমিকা বোঝা
- একটি সহজ সংখ্যাগত উদাহরণের মাধ্যমে Artificial Neuron-এর হিসাব করতে পারা

মূল ধারণা (Main Concept)

৫.১ Artificial Neuron-এর গঠন

একটি Artificial Neuron একাধিক Input গ্রহণ করে, প্রতিটি Input-কে একটি নির্দিষ্ট Weight দিয়ে গুণ করে, সব গুণফল যোগ করে, এবং তার সাথে একটি Bias যোগ করে। একে গাণিতিকভাবে প্রকাশ করা হয়:

$$z = \sum_i w_i x_i + b$$

এরপর এই ফলাফল z -কে একটি **Activation Function** f -এর মধ্য দিয়ে পাঠানো হয়, যা চূড়ান্ত output নির্ধারণ করে:

$$a = f(z)$$

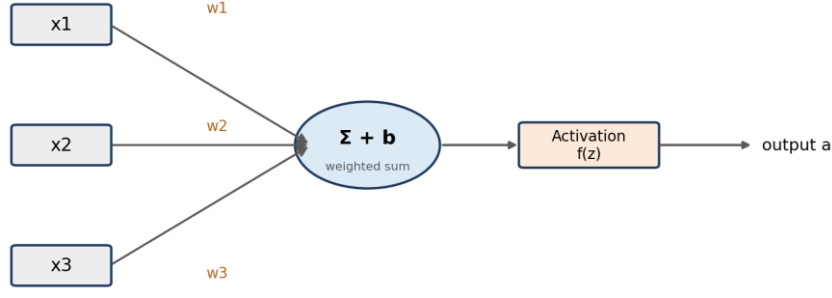
৫.২ প্রতিটি উপাদানের ব্যাখ্যা

এখানে x হলো **Input** (data থেকে পাওয়া মান), w হলো **Weight** (প্রতিটি input-এর গুরুত্ব নির্ধারণ করে), এবং b হলো **Bias** (একটি অতিরিক্ত সমন্বয়কারী মান)। z -কে **Weighted Sum** বলা হয়, এবং Weighted Sum-এর পরে Activation Function ব্যবহার করে neuron-এর চূড়ান্ত output a নির্ধারণ করা হয়।

Activation Function ছাড়া একটি Neural Network, তার layer-সংখ্যা যত বেশিই হোক না কেন, কার্যত একটি সরল রৈখিক (linear) ফাংশনের মতো আচরণ করত। Activation Function network-এ **Non-linearity** যোগ করে, যা জটিল প্যাটার্ন শেখার জন্য অপরিহার্য।

Key Terms

Input • Weight • Bias • Weighted Sum • Activation Function • Output • Non-linearity



চিত্র ৫.১: Artificial Neuron-এর মৌলিক গঠন

উদাহরণ ৫.১: একটি সহজ Neuron হিসাব

ধরা যাক একটি neuron-এর দুটি input রয়েছে: $x_1 = 2$, $x_2 = 3$, তাদের সংশ্লিষ্ট weight $w_1 = 0.5$, $w_2 = -1$, এবং bias $b = 1$

Weighted Sum: $z = (0.5 \times 2) + (-1 \times 3) + 1 = 1 - 3 + 1 = -1$

যদি Activation Function হিসেবে একটি সরল threshold function ব্যবহার করা হয় (যেখানে $z \geq 0$ হলে output 1, নাহলে 0), তাহলে যেহেতু $z = -1 < 0$, তাই output $a = 0$ হবে।

Key Takeaways

- একটি Artificial Neuron input, weight ও bias ব্যবহার করে একটি Weighted Sum গণনা করে, তারপর Activation Function প্রয়োগ করে।
- Weight নির্ধারণ করে প্রতিটি input কতটা প্রভাব ফেলবে; Bias সিদ্ধান্তের সীমারেখা সমন্বয় করে।
- Activation Function network-এ Non-linearity যোগ করে, যা জটিল প্যাটার্ন শেখার জন্য অপরিহার্য।

Quick Review Questions

16. $z = \sum_i w_i x_i + b$ সমীকরণের প্রতিটি প্রতীক ব্যাখ্যা করুন।
17. Activation Function ছাড়া Neural Network-এর সীমাবদ্ধতা কী হতো?
18. উদাহরণ ৫.১-এ $w_1 = 1$ হলে output-এর ফলাফল কী হতো, হিসাব করুন।

PART IV

Perceptron

Chapter 6

Perceptron

Perceptron

Perceptron হলো সবচেয়ে সরল Artificial Neuron মডেল, যা Input, Weight, Bias এবং একটি Activation Function ব্যবহার করে Binary Classification সম্পন্ন করে।

Learning Objectives

- Perceptron-এর সংজ্ঞা ও গঠন ব্যাখ্যা করতে পারা
- Weight ও Bias-এর প্রভাব বিশদভাবে বোঝা
- Perceptron কীভাবে Binary Classification করে তা ব্যাখ্যা করতে পারা

মূল ধারণা (Main Concept)

৬.১ Perceptron কী?

Perceptron হলো সবচেয়ে সরল Artificial Neuron মডেল, যা মূলত **Binary Classification**-এর জন্য ব্যবহৃত হয়—অর্থাৎ, input-কে দুটি শ্রেণির (যেমন হ্যাঁ/না, ০/১) একটিতে শ্রেণীবদ্ধ করা। এটিকে একটি একক neuron হিসেবে দেখা যেতে পারে, যা input নেয়, Weighted Sum গণনা করে, এবং Activation Function-এর মাধ্যমে চূড়ান্ত সিদ্ধান্ত দেয়।

৬.২–৬.৫ Input, Weight, Bias এবং Activation

Perceptron-এর মূল উপাদানগুলো Chapter ৫-এ আলোচিত Artificial Neuron-এর মতোই: Input (x), Weight (w), Bias (b), এবং Activation। Perceptron-এর ক্ষেত্রে Activation Function সাধারণত একটি step function হয়, যা output-কে সরাসরি ০ অথবা ১-এ রূপান্তর করে।

৬.৬ Binary Classification

Perceptron একটি **Decision Boundary** তৈরি করে, যা input space-কে দুটি অংশে ভাগ করে—একটি অংশ একটি শ্রেণির (class) জন্য, অন্যটি ভিন্ন শ্রেণির জন্য। যদি একটি নতুন input এই boundary-র এক পাশে পড়ে, তাহলে তাকে এক শ্রেণিতে এবং অন্য পাশে পড়লে অন্য শ্রেণিতে ভাগ করা হয়।

Weight বিস্তারিত

Weight-এর মান positive বা negative হতে পারে। একটি **Positive Weight** নির্দেশ করে যে সংশ্লিষ্ট input বাড়লে output-এর দিকে ইতিবাচক প্রভাব পড়ে, আর **Negative Weight** নির্দেশ করে বিপরীত প্রভাব। Weight-এর মানের **Magnitude** (মাত্রা) যত বড়, সেই input-এর সিদ্ধান্তের উপর প্রভাব তত বেশি। ছোট magnitude-এর weight নির্দেশ করে সংশ্লিষ্ট input-এর প্রভাব তুলনামূলক কম।

Bias বিস্তারিত

শুধু Weight ও Input দিয়ে গঠিত Weighted Sum সবসময় শূন্য বিন্দু দিয়ে যাওয়া একটি decision boundary তৈরি করে, যা বাস্তব সমস্যায় প্রায়ই অপরিপূর্ণ। **Bias** এই boundary-কে প্রয়োজন অনুযায়ী স্থানান্তরিত (shift) করতে সাহায্য করে, ফলে model আরও নমনীয়ভাবে বিভিন্ন প্যাটার্নে মানিয়ে নিতে পারে।

Key Terms

Perceptron • Binary Classification • Decision Boundary • Positive Weight • Negative Weight

উদাহরণ ৬.১: Weight ও Bias-এর প্রভাব

ধরা যাক একটি সাধারণ email-spam শনাক্তকারী perceptron-এ 'বিনামূল্যে' শব্দের উপস্থিতি একটি input feature। যদি এই feature-এর weight বড় positive মান হয় (যেমন $w = 3$), তাহলে এই শব্দটি থাকলে email-কে spam হিসেবে চিহ্নিত করার সম্ভাবনা জোরালোভাবে বেড়ে যাবে। যদি weight ছোট হয় (যেমন $w = 0.1$), তাহলে এই feature-এর প্রভাব হবে সামান্য।

Key Takeaways

- Perceptron হলো সবচেয়ে সরল Artificial Neuron মডেল, যা মূলত Binary Classification-এর জন্য ব্যবহৃত হয়।
- Weight-এর চিহ্ন (positive/negative) এবং মাত্রা নির্ধারণ করে input-এর প্রভাবের দিক ও তীব্রতা।
- Bias decision boundary-কে প্রয়োজন অনুযায়ী স্থানান্তরিত করতে সাহায্য করে।

Quick Review Questions

19. Perceptron কীভাবে Binary Classification সম্পন্ন করে?
20. Positive Weight এবং Negative Weight-এর মধ্যে পার্থক্য কী?
21. Bias না থাকলে model-এর কী সীমাবদ্ধতা দেখা দিতে পারে?

Chapter 7

Single-Layer Perceptron এবং Linear Separability

Learning Objectives

- Single-Layer Perceptron-এর কার্যপদ্ধতি বোঝা
- Linear Separability-র ধারণা ব্যাখ্যা করতে পারা
- AND, OR এবং XOR সমস্যার মধ্যে পার্থক্য বিশ্লেষণ করতে পারা

মূল ধারণা (Main Concept)

৭.১–৭.৩ Single-Layer Perceptron, Decision Boundary ও Linear Separability

একটি **Single-Layer Perceptron** হলো একটিমাত্র স্তরের perceptron, যা সরাসরি input থেকে output গণনা করে। এটি একটি সরলরৈখিক (linear) Decision Boundary তৈরি করতে পারে। যদি কোনো dataset-এর দুটি শ্রেণিকে একটিমাত্র সরলরেখা (বা উচ্চমাত্রিক ক্ষেত্রে একটি hyperplane) দিয়ে সম্পূর্ণভাবে আলাদা করা যায়, তাহলে সেই dataset-কে **Linearly Separable** বলা হয়।

৭.৪–৭.৬ AND, OR এবং XOR

নিচের truth table তিনটি সাধারণ logical operation দেখায়:

কেন AND ও OR পৃথকযোগ্য, কিন্তু XOR নয়

চিত্র ৭.১-এ দেখা যায়, AND ও OR-এর ক্ষেত্রে একটি একক সরলরেখা দিয়ে '১' ফলাফলযুক্ত বিন্দুগুলোকে '০' ফলাফলযুক্ত বিন্দু থেকে সম্পূর্ণভাবে আলাদা করা সম্ভব। কিন্তু XOR-এর ক্ষেত্রে '১' ফলাফলযুক্ত বিন্দু দুটি কর্ণ বরাবর অবস্থিত এবং '০' ফলাফলযুক্ত বিন্দু দুটি বিপরীত কর্ণ বরাবর অবস্থিত—ফলে কোনো একক সরলরেখা দিয়ে এদের আলাদা করা সম্ভব নয়। এই কারণে একটি Single-Layer Perceptron কখনোই XOR সমস্যার সঠিক সমাধান শিখতে পারে না। এই সীমাবদ্ধতা ইতিহাসে **XOR Problem** নামে পরিচিত, এবং এটিই বহুস্তর Neural Network-এর প্রয়োজনীয়তার একটি মূল কারণ।

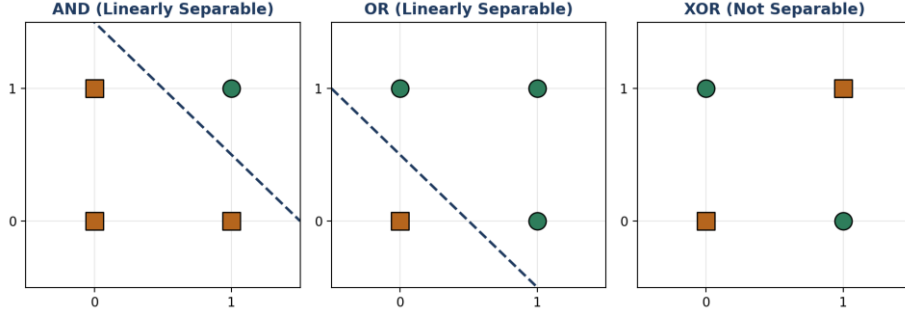
Key Terms

Single-Layer Perceptron • Decision Boundary • Linear Separability • AND • OR • XOR

সারণি ৭.১: AND, OR ও XOR-এর Truth Table

x1	x2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1

x1	x2	AND	OR	XOR
1	1	1	1	0



চিত্র ৭.১: AND ও OR সরলরেখা দ্বারা পৃথকযোগ্য (Linearly Separable), কিন্তু XOR নয়

Key Takeaways

- Single-Layer Perceptron শুধুমাত্র Linearly Separable সমস্যার সমাধান করতে পারে।
- AND ও OR সমস্যা Linearly Separable, তাই Single-Layer Perceptron এগুলো সমাধান করতে পারে।
- XOR সমস্যা Linearly Separable নয়, তাই এটি Single-Layer Perceptron দিয়ে সমাধান করা যায় না।
- XOR Problem-ই মূলত বহুস্তর Neural Network উদ্ভাবনের একটি গুরুত্বপূর্ণ কারণ।

Quick Review Questions

22. Linear Separability বলতে কী বোঝায়?
23. AND ও OR-এর Truth Table কেন Linearly Separable, তা ব্যাখ্যা করুন।
24. XOR সমস্যা কেন Single-Layer Perceptron দিয়ে সমাধান করা যায় না?

PART V

মাল্টি-লেয়ার নিউরাল নেটওয়ার্ক

Chapter 8

কেন একাধিক Layer প্রয়োজন

Learning Objectives

- Single Layer-এর সীমাবদ্ধতা ব্যাখ্যা করতে পারা
- একাধিক Layer কীভাবে Non-linear সমস্যার সমাধান করে তা বোঝা
- Multi-Layer Perceptron (MLP)-এর ধারণা ব্যাখ্যা করতে পারা

মূল ধারণা (Main Concept)

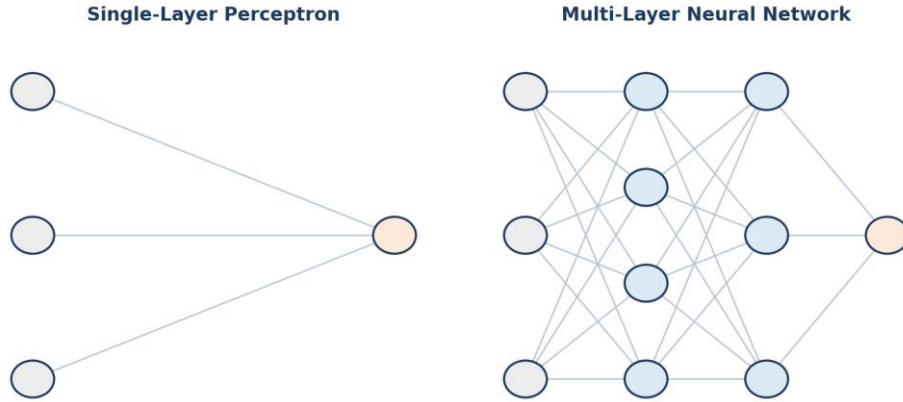
৮.১ সীমাবদ্ধতা থেকে সমাধান

পূর্ববর্তী অধ্যায়ে দেখা গেছে যে একটি Single-Layer Perceptron শুধুমাত্র Linearly Separable সমস্যার সমাধান করতে পারে, যা XOR-এর মতো সমস্যায় ব্যর্থ হয়। এই সীমাবদ্ধতা কাটিয়ে ওঠার জন্য একাধিক layer যুক্ত করে **Multiple Layer Perceptron (MLP)** তৈরি করা হয়।

একাধিক layer ব্যবহার করলে network জটিল, Non-linear decision boundary তৈরি করতে পারে, যা শুধুমাত্র সরলরেখা দিয়ে সম্ভব নয় এমন প্যাটার্নও শনাক্ত করতে সক্ষম। প্রতিটি অতিরিক্ত layer network-কে আরও জটিল সিদ্ধান্তের সীমারেখা গঠন করতে সাহায্য করে।

Key Terms

Multiple Layer Perceptron (MLP) • Non-linear Representation



চিত্র ৮.১: Single-Layer Perceptron বনাম Multi-Layer Neural Network

Key Takeaways

- Single Layer শুধুমাত্র linear সীমারেখা তৈরি করতে পারে, যা XOR-এর মতো সমস্যায় অপার্যাপ্ত।
- একাধিক layer যুক্ত করলে network জটিল, non-linear decision boundary তৈরি করতে সক্ষম হয়।
- একাধিক layer বিশিষ্ট network-কে Multiple Layer Perceptron (MLP) বলা হয়।

Quick Review Questions

25. কেন একটি Single Layer XOR সমস্যার সমাধান করতে পারে না?
26. একাধিক layer কীভাবে Non-linear decision boundary তৈরি করতে সাহায্য করে?

Chapter 9

Input Layer, Hidden Layers এবং Output Layer

Learning Objectives

- Input, Hidden এবং Output Layer-এর ভূমিকা ব্যাখ্যা করতে পারা
- Layer-ভিত্তিক feature learning-এর ধারণা বোঝা
- একটি image-classification উদাহরণের মাধ্যমে layer-এর কাজ বিশ্লেষণ করতে পারা

মূল ধারণা (Main Concept)

Input Layer

Input Layer হলো network-এর প্রথম স্তর, যা raw data থেকে পাওয়া feature-গুলো গ্রহণ করে। এই layer কোনো গণনা করে না; এটি শুধু data-কে network-এর ভেতরে প্রবেশ করায়।

Hidden Layer

Hidden Layer হলো Input Layer ও Output Layer-এর মাঝখানে অবস্থিত স্তর, যেখানে প্রকৃত শেখার প্রক্রিয়া ঘটে। প্রতিটি Hidden Layer আগের layer থেকে পাওয়া তথ্যকে রূপান্তরিত (transform) করে ক্রমশ বেশি উপযোগী representation তৈরি করে।

Output Layer

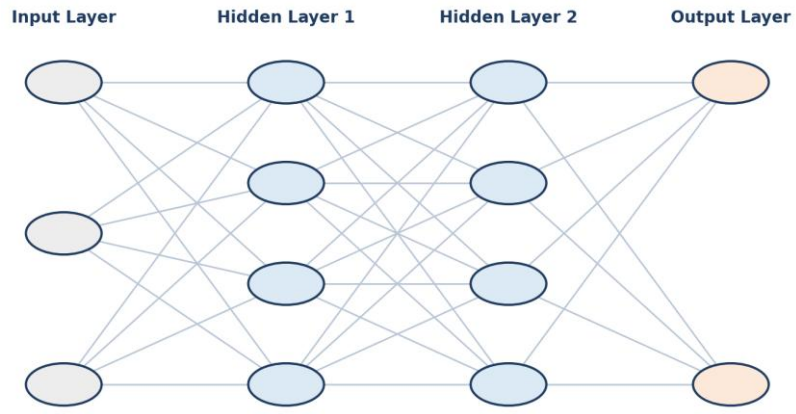
Output Layer হলো network-এর শেষ স্তর, যা চূড়ান্ত prediction তৈরি করে—যেমন একটি classification সমস্যায় কোন শ্রেণিতে input পড়ে তা নির্ধারণ করা।

Layer-ভিত্তিক প্যাটার্ন শেখা

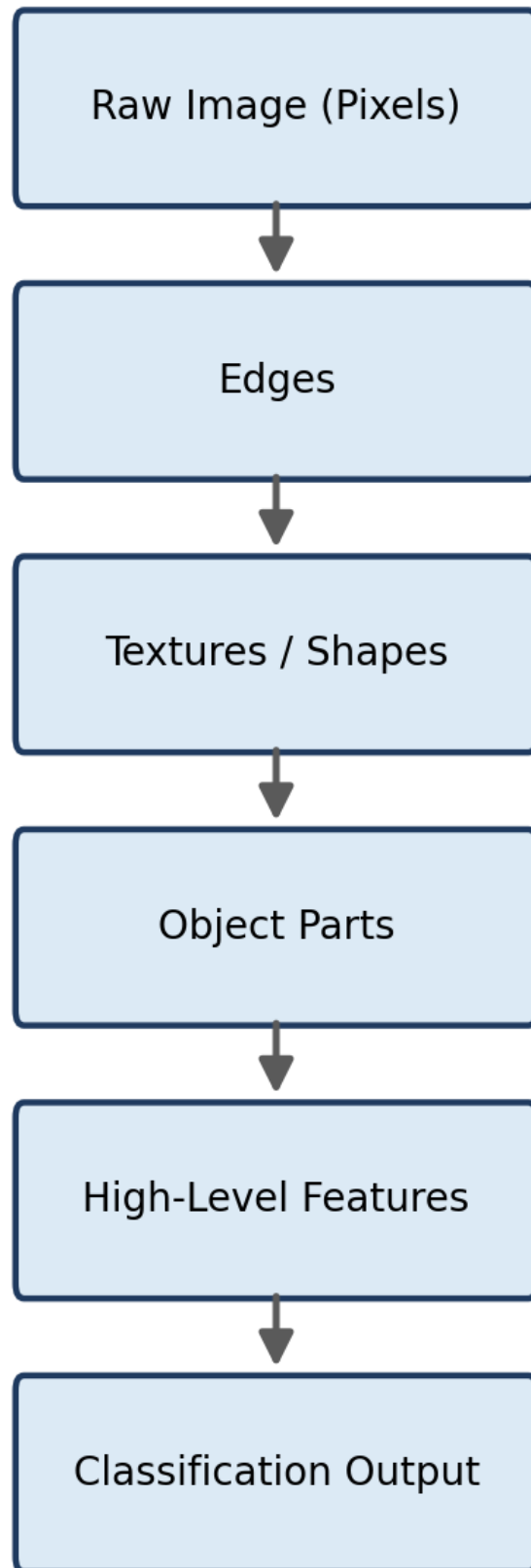
একটি image-classification network-এ সাধারণত নিম্নলিখিত ধারা লক্ষ্য করা যায়: প্রাথমিক (early) layer সরল প্যাটার্ন যেমন edge শনাক্ত করে; মধ্যবর্তী (middle) layer সেই সরল প্যাটার্নগুলোর সমন্বয় শনাক্ত করে; গভীরতর (deeper) layer আরও জটিল ও উচ্চ-স্তরের প্যাটার্ন শনাক্ত করে; এবং শেষ layer কাজ-নির্দিষ্ট (task-specific) prediction তৈরি করে।

Key Terms

Input Layer • Hidden Layer • Output Layer



চিত্র ৯.১: Input Layer, Hidden Layer ও Output Layer-এর গঠন



চিত্র ৯.২: একটি image-classification network-এ ধাপে ধাপে জটিলতর feature শেখা

Key Takeaways

- Input Layer raw feature গ্রহণ করে, কোনো গণনা করে না।

- Hidden Layer-এ প্রকৃত শেখার প্রক্রিয়া ঘটে, প্রতিটি layer ক্রমশ বেশি উপযোগী representation তৈরি করে।
- Output Layer চূড়ান্ত prediction তৈরি করে।
- প্রাথমিক layer সরল প্যাটার্ন, গভীরতর layer জটিল ও উচ্চ-স্তরের প্যাটার্ন শেখে।

Quick Review Questions

27. Input Layer, Hidden Layer এবং Output Layer-এর কাজ ব্যাখ্যা করুন।
28. একটি image-classification network-এ early layer ও deep layer-এর শেখা প্যাটার্নের মধ্যে পার্থক্য কী?

PART VI

Deep Learning কীভাবে শেখে

Chapter 10

Learnable Parameters

Learning Objectives

- Learnable Parameters-এর সংজ্ঞা ব্যাখ্যা করতে পারা
- Weight ও Bias কেন 'learnable' তা বোঝা
- Training কীভাবে parameter-এর মান পরিবর্তন করে তা ব্যাখ্যা করতে পারা

মূল ধারণা (Main Concept)

Parameter কী?

Parameter হলো model-এর ভেতরের এমন মান, যা model-এর আচরণ নির্ধারণ করে। Deep Learning-এ প্রতিটি neuron-এর Weight ও Bias-ই মূল parameter। এই সম্পর্কটি নিম্নলিখিতভাবে সংক্ষেপে প্রকাশ করা যায়:

কেন 'Learnable'

এই parameter-গুলোকে 'learnable' বলা হয় কারণ তাদের মান manually নির্ধারণ করা হয় না; বরং training প্রক্রিয়ার মাধ্যমে data থেকে স্বয়ংক্রিয়ভাবে নির্ধারিত হয়। একটি বড় Deep Learning model-এ লক্ষ লক্ষ বা কোটি কোটি parameter থাকতে পারে, যা মানুষের পক্ষে হাতে নির্ধারণ করা বাস্তবসম্মত নয়।

Training শুরু হয় সাধারণত এলোমেলো (random) মান দিয়ে, এবং প্রতিটি training ধাপে Optimization প্রক্রিয়ার মাধ্যমে এই মান ধীরে ধীরে সমন্বয় (adjust) করা হয়, যাতে model-এর ভুল ক্রমশ কমে আসে।

Key Terms

Parameter • Learnable Parameters

$$\text{Learnable Parameters} = \text{Weights} + \text{Biases}$$

Key Takeaways

- Weight ও Bias-ই Deep Learning model-এর মূল Learnable Parameters।
- এই parameter-গুলোর মান মানুষ হাতে নির্ধারণ করে না; training-এর মাধ্যমে data থেকে স্বয়ংক্রিয়ভাবে শেখা হয়।
- Training সাধারণত random মান দিয়ে শুরু হয়, এবং ধীরে ধীরে Optimization-এর মাধ্যমে সঠিক মানের দিকে অগ্রসর হয়।

Quick Review Questions

29. Learnable Parameters বলতে কী বোঝায়?

30. কেন মানুষ সরাসরি প্রতিটি parameter হাতে নির্ধারণ করে না?

Chapter 11

Forward Propagation

Learning Objectives

- Forward Propagation-এর সম্পূর্ণ প্রক্রিয়া ব্যাখ্যা করতে পারা
- Input থেকে Prediction পর্যন্ত তথ্য প্রবাহ বুঝতে পারা

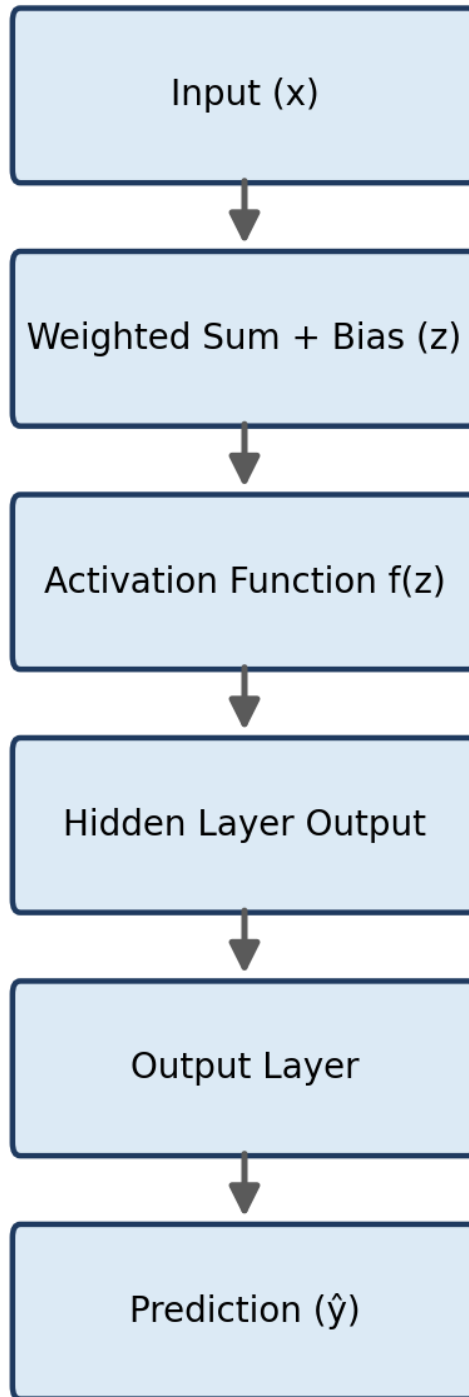
মূল ধারণা (Main Concept)

Forward Propagation-এর প্রবাহ

Forward Propagation হলো সেই প্রক্রিয়া, যার মাধ্যমে input data network-এর মধ্য দিয়ে স্তরে স্তরে অগ্রসর হয়ে চূড়ান্ত prediction তৈরি করে। প্রতিটি layer-এ Weighted Sum গণনা করা হয়, তারপর Activation Function প্রয়োগ করা হয়, এবং ফলাফল পরবর্তী layer-এ পাঠানো হয়। এই প্রক্রিয়া Output Layer পর্যন্ত চলতে থাকে, যেখানে চূড়ান্ত prediction পাওয়া যায়।

Key Terms

Forward Propagation



চিত্র ১১.১: Forward Propagation-এর প্রবাহ

উদাহরণ ১১.১: একটি সহজ Forward Pass

ধরা যাক একটি ছোট network-এ একটি input $x = 1$, weight $w = 2$, এবং bias $b = 0$ । Weighted Sum: $z = (2 \times 1) + 0 = 2$ । যদি Activation Function হিসেবে একটি সরল identity function ($f(z) = z$) ব্যবহার করা হয়, তাহলে output $a = 2$, যা এই একক-neuron network-এর চূড়ান্ত prediction।

Key Takeaways

- Forward Propagation-এ data Input Layer থেকে Output Layer পর্যন্ত স্তরে স্তরে অগ্রসর হয়।

- প্রতিটি layer-এ Weighted Sum ও Activation Function প্রয়োগ করা হয়।
- Forward Propagation-এর চূড়ান্ত ফলাফলই model-এর prediction।

Quick Review Questions

31. Forward Propagation-এর ধাপগুলো ক্রমানুসারে লিখুন।
32. উদাহরণ ১১.১-এ $b = 1$ হলে চূড়ান্ত output কত হতো, হিসাব করুন।

Chapter 12

Loss Function

Learning Objectives

- Loss Function-এর প্রয়োজনীয়তা ব্যাখ্যা করতে পারা
- Actual Target ও Prediction-এর মধ্যে সম্পর্ক বোঝা
- সাধারণ Loss Function-এর নাম ও ধারণাগত ব্যবহার জানা

মূল ধারণা (Main Concept)

কেন Loss প্রয়োজন

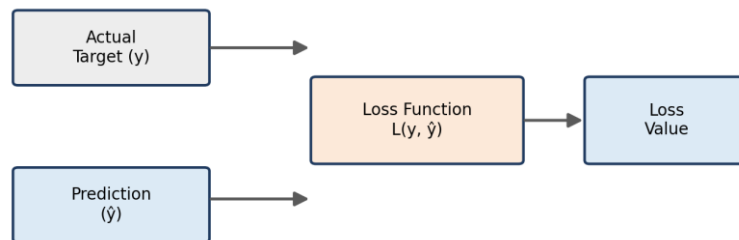
Training-এর সময় model-এর Prediction ($\hat{y}$) সবসময় Actual Target (y)-এর সাথে সঠিকভাবে মেলে না। এই পার্থক্যকে সংখ্যাগতভাবে পরিমাপ করার জন্য একটি **Loss Function** ব্যবহার করা হয়, যা model কতটা ভুল করছে তার একটি একক সংখ্যাগত পরিমাপ প্রদান করে। Loss-এর মান যত কম, model-এর prediction তত সঠিক।

সাধারণ Loss Function

সংখ্যাগত (regression) সমস্যার জন্য **Mean Squared Error (MSE)** সাধারণত ব্যবহৃত হয়, যেখানে prediction ও target-এর পার্থক্যের বর্গের গড় নেওয়া হয়। Binary classification-এর জন্য **Binary Cross Entropy** এবং Multiclass classification-এর জন্য **Cross Entropy** ব্যবহৃত হয়। এই অধ্যায়ে এদের গাণিতিক derivation নিয়ে আলোচনা করা হয়নি; শুধু ধারণাগতভাবে বোঝানো হয়েছে যে এগুলো model-এর ভুলকে বিভিন্নভাবে পরিমাপ করে।

Key Terms

Loss Function • Mean Squared Error • Binary Cross Entropy • Cross Entropy



চিত্র ১২.১: Actual Target ও Prediction থেকে Loss গণনা

Key Takeaways

- Loss Function model-এর prediction ও actual target-এর মধ্যকার পার্থক্য সংখ্যাগতভাবে পরিমাপ করে।
- Loss-এর মান যত কম, model তত সঠিক।

- বিভিন্ন সমস্যার জন্য বিভিন্ন Loss Function ব্যবহৃত হয়—যেমন regression-এ MSE, classification-এ Cross Entropy।

Quick Review Questions

33. Loss Function-এর মূল উদ্দেশ্য কী?
34. Mean Squared Error এবং Cross Entropy কোন ধরনের সমস্যায় ব্যবহৃত হয়?

Chapter 13

Backpropagation

Backpropagation

Backpropagation হলো Loss থেকে gradient গণনা করে network-এর parameters কীভাবে পরিবর্তন করতে হবে তা নির্ধারণের learning mechanism।

Learning Objectives

- Backpropagation-এর মূল ধারণা ব্যাখ্যা করতে পারা
- Gradient-এর অর্থ ব্যাখ্যা করতে পারা
- Chain Rule-এর ধারণাগত ভূমিকা বোঝা

মূল ধারণা (Main Concept)

১৩.১ Backpropagation কী?

Backpropagation হলো সেই পদ্ধতি, যার মাধ্যমে network Loss-এর মান থেকে শুরু করে output layer থেকে input layer-এর দিকে পিছনের দিকে গণনা করে প্রতিটি Weight ও Bias কতটা পরিবর্তন করা প্রয়োজন তা নির্ধারণ করে।

১৩.২ কেন Backpropagation প্রয়োজন

যেহেতু একটি Deep Learning model-এ অসংখ্য Learnable Parameters থাকে, তাই প্রতিটি parameter Loss-কে কতটা প্রভাবিত করছে তা efficient ভাবে গণনা করার একটি পদ্ধতি প্রয়োজন। Backpropagation এই কাজ সুশৃঙ্খলভাবে সম্পন্ন করে, layer অনুযায়ী পিছনের দিকে গণনা এগিয়ে নিয়ে।

১৩.৩–১৩.৪ Error Contribution ও Gradient

প্রতিটি parameter Loss-এ কতটা 'অবদান' রাখছে তা বোঝার জন্য **Gradient** ব্যবহার করা হয়। Gradient হলো একটি পরিমাপ, যা নির্দেশ করে একটি নির্দিষ্ট parameter সামান্য পরিবর্তন করলে Loss কতটা পরিবর্তিত হবে।

১৩.৫ Chain Rule ধারণা

যেহেতু একটি Deep Network-এ output অনেকগুলো layer-এর মধ্য দিয়ে গণনা হয়ে আসে, তাই Loss-এর উপর একটি নির্দিষ্ট প্রাথমিক layer-এর parameter-এর প্রভাব বোঝার জন্য layer-বাই-layer প্রভাবগুলো ধারাবাহিকভাবে গুণ করা প্রয়োজন হয়। গণিতে এই ধারণাকে **Chain Rule** বলা হয়। এই বইয়ের এর বিস্তারিত calculus derivation আলোচনা করা হয়নি; শুধু ধারণাগতভাবে বোঝানো হয়েছে।

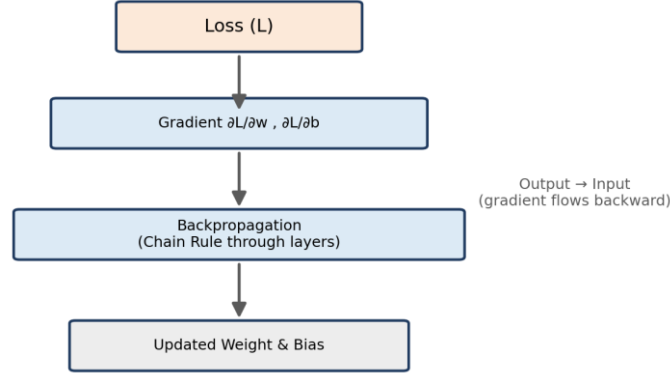
১৩.৬–১৩.৭ Weight ও Bias Gradient

প্রতিটি Weight-এর জন্য একটি পৃথক gradient গণনা করা হয়, যাকে বলা হয় $(\partial L)/(\partial w)$, এবং একইভাবে প্রতিটি Bias-এর জন্য $(\partial L)/(\partial b)$ । এখানে $(\partial L)/(\partial w)$ -কে ধারণাগতভাবে ব্যাখ্যা করা যায়:

"এই Weight পরিবর্তন করলে Loss কতটা পরিবর্তিত হয় তার পরিমাপ।"

Key Terms

Backpropagation • Gradient • Chain Rule



চিত্র ১৩.১: Backpropagation-এ Loss থেকে Gradient পর্যন্ত পিছনের দিকে গণনা

সহজ উপলব্ধি

একটি ঢালু পাহাড়ে হাঁটার কথা কল্পনা করুন, যেখানে লক্ষ্য হলো সবচেয়ে নিচু বিন্দুতে (সর্বনিম্ন Loss) পৌঁছানো। Gradient আপনাকে বলে দেয় বর্তমান অবস্থানে ঢাল কোন দিকে এবং কতটা খাড়া—এই তথ্য ব্যবহার করেই পরবর্তী পদক্ষেপের দিক নির্ধারণ করা হয়।

Key Takeaways

- Backpropagation Loss থেকে শুরু করে output থেকে input-এর দিকে পিছনে গিয়ে প্রতিটি parameter-এর gradient গণনা করে।
- Gradient নির্দেশ করে একটি parameter পরিবর্তন করলে Loss কতটা পরিবর্তিত হবে।
- Chain Rule ব্যবহার করে layer-বাই-ayer প্রভাব ধারাবাহিকভাবে গণনা করা হয়।

Quick Review Questions

35. Backpropagation কেন প্রয়োজন?
36. Gradient-এর ধারণাগত অর্থ নিজের ভাষায় ব্যাখ্যা করুন।
37. Chain Rule Backpropagation-এ কীভাবে ব্যবহৃত হয়?

Chapter 14

Optimization

Learning Objectives

- Optimizer ও Gradient Descent-এর ধারণা ব্যাখ্যা করতে পারা
- Learning Rate-এর ভূমিকা বোঝা
- সম্পূর্ণ Training Cycle বর্ণনা করতে পারা

মূল ধারণা (Main Concept)

১৪.১ Optimizer

Optimizer হলো সেই algorithm, যা Gradient ব্যবহার করে Weight ও Bias-এর মান হালনাগাদ (update) করে, যাতে Loss ধীরে ধীরে কমতে থাকে।

১৪.২ Gradient Descent

Gradient Descent হলো সবচেয়ে মৌলিক Optimization পদ্ধতি, যেখানে প্রতিটি parameter-কে তার gradient-এর বিপরীত দিকে সামান্য পরিমাণে সরানো হয়—কারণ gradient-এর বিপরীত দিকে গেলে Loss কমার সম্ভাবনা থাকে।

১৪.৩ Learning Rate

Learning Rate (η) নির্ধারণ করে প্রতিটি ধাপে parameter কতটা পরিমাণে পরিবর্তিত হবে। খুব বড় Learning Rate ব্যবহার করলে training অস্থির হয়ে যেতে পারে, আবার খুব ছোট Learning Rate ব্যবহার করলে training অত্যন্ত ধীর হয়ে যায়।

১৪.৪ Parameter Update

Weight হালনাগাদ করার সাধারণ সূত্রটি নিম্নরূপ:

$$w_{new} = w_{old} - \eta (\partial L) / (\partial w)$$

এখানে w_{old} হলো বর্তমান Weight, η হলো Learning Rate, এবং $(\partial L) / (\partial w)$ হলো সেই Weight-এর Gradient। একইভাবে Bias-ও হালনাগাদ করা হয়।

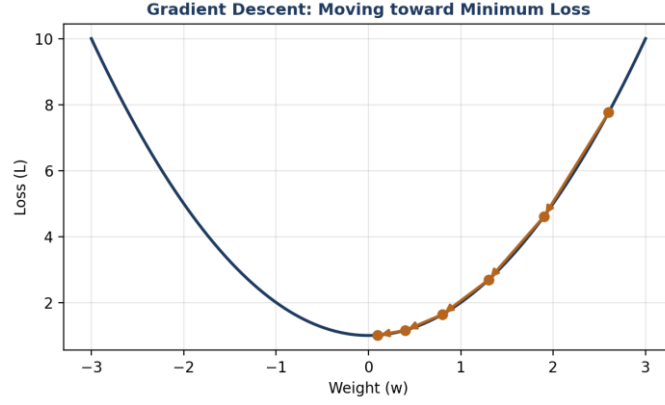
সম্পূর্ণ Training Cycle

একটি সম্পূর্ণ training ধাপে নিম্নলিখিত প্রক্রিয়া ঘটে: Input থেকে Forward Pass, তারপর Prediction, তারপর Loss গণনা, তারপর Backpropagation-এর মাধ্যমে Gradient গণনা, তারপর Optimizer-এর মাধ্যমে Weight ও Bias হালনাগাদ, এবং এই সম্পূর্ণ চক্র বহুবার পুনরাবৃত্তি (repeat) করা হয়।

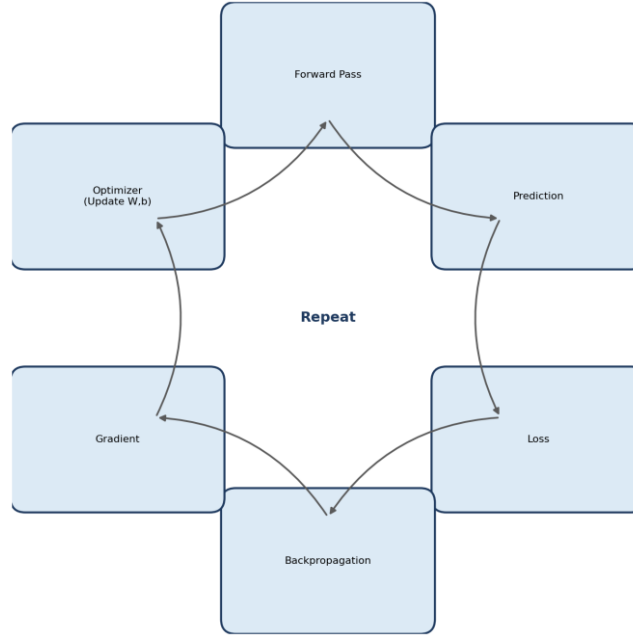
সংক্ষেপে বলা যায়, Deep Learning মূলত বারবার Optimization-এর মাধ্যমে উপযোগী parameter মান শেখার একটি প্রক্রিয়া।

Key Terms

Optimizer • Gradient Descent • Learning Rate • Parameter Update



চিত্র ১৪.১: Gradient Descent-এর মাধ্যমে সর্বনিম্ন Loss-এর দিকে অগ্রসর হওয়া



চিত্র ১৪.২: সম্পূর্ণ Training Cycle

Key Takeaways

- Optimizer Gradient ব্যবহার করে Weight ও Bias হালনাগাদ করে, যাতে Loss কমে।
- Learning Rate নির্ধারণ করে প্রতিটি ধাপে parameter কতটা পরিবর্তিত হবে।
- সম্পূর্ণ Training Cycle: Forward Pass → Prediction → Loss → Backpropagation → Gradient → Optimizer → Update → পুনরাবৃত্তি।
- Deep Learning মূলত বারবার Optimization-এর মাধ্যমে উপযোগী parameter মান শেখার একটি প্রক্রিয়া।

Quick Review Questions

38. Learning Rate খুব বড় বা খুব ছোট হলে কী সমস্যা হতে পারে?

39. $w_{new} = w_{old} - \eta (\partial L / \partial w)$ সূত্রের প্রতিটি অংশ ব্যাখ্যা করুন।
40. সম্পূর্ণ Training Cycle-এর ধাপগুলো ক্রমানুসারে লিখুন।

PART VII

হার্ডওয়ার কেন গুরুত্বপূর্ণ

Chapter 15

CPU এবং GPU

Learning Objectives

- CPU ও GPU-এর মূল পার্থক্য ব্যাখ্যা করতে পারা
- Parallelism-এর ধারণা বোঝা
- Deep Learning কেন Parallelism থেকে উপকৃত হয় তা ব্যাখ্যা করতে পারা

মূল ধারণা (Main Concept)

১৫.১ CPU

CPU (Central Processing Unit) সাধারণত অল্প সংখ্যক কিন্তু অত্যন্ত শক্তিশালী core নিয়ে গঠিত, যা জটিল ও ধারাবাহিক (sequential) কাজ দ্রুত সম্পন্ন করতে দক্ষ।

১৫.২–১৫.৩ GPU ও Graphics Card

GPU (Graphics Processing Unit), যা সাধারণত একটি **Graphics Card**-এ বসানো থাকে, হাজার হাজার ছোট core নিয়ে গঠিত। এই core-গুলো প্রতিটি পৃথকভাবে শক্তিশালী না হলেও, একসাথে বহু সহজ গণনা সমান্তরালভাবে সম্পন্ন করতে পারে।

১৫.৪ Parallelism

Parallelism বলতে বোঝায় একই সময়ে একাধিক গণনা সম্পন্ন করার ক্ষমতা। Deep Learning-এর মূল গণনা—matrix multiplication এবং tensor operation—স্বভাবতই এমনভাবে গঠিত যে সেগুলোকে বহু ছোট ছোট স্বাধীন গণনায় ভাগ করা যায়, যা GPU-র সমান্তরাল স্থাপত্যের সাথে অত্যন্ত উপযোগী।

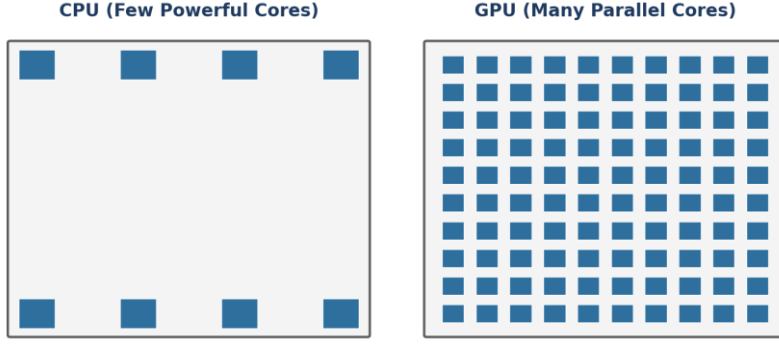
১৫.৫ Deep Learning কেন Parallelism থেকে উপকৃত হয়

একটি Neural Network-এর প্রতিটি layer-এর গণনা মূলত বৃহৎ matrix-এর গুণফল। GPU একসাথে হাজার হাজার এই ধরনের ছোট গণনা সম্পন্ন করতে পারায়, বড় Deep Learning model প্রশিক্ষণের সময় উল্লেখযোগ্যভাবে কমে আসে।

এখানে একটি গুরুত্বপূর্ণ বিষয় স্পষ্ট করা প্রয়োজন: CPU দিয়েও Deep Learning চালানো সম্ভব, কিন্তু বড় ও অত্যন্ত সমান্তরাল কাজের জন্য GPU সাধারণত অনেক বেশি efficient।

Key Terms

CPU • GPU • Graphics Card • Parallelism



চিত্র ১৫.১: CPU (অল্প শক্তিশালী core) বনাম GPU (বহু সমান্তরাল core)

Key Takeaways

- CPU-তে অল্প সংখ্যক শক্তিশালী core থাকে; GPU-তে হাজার হাজার সরল, সমান্তরাল core থাকে।
- Parallelism মানে একইসাথে একাধিক গণনা সম্পন্ন করা।
- Deep Learning-এর matrix multiplication ও tensor operation GPU-র সমান্তরাল কাঠামোর সাথে বিশেষভাবে উপযোগী।
- CPU দিয়ে Deep Learning চালানো সম্ভব, কিন্তু বড় কাজের জন্য GPU সাধারণত অনেক বেশি efficient।

Quick Review Questions

41. CPU ও GPU-এর মূল architectural পার্থক্য কী?
42. Deep Learning-এর কোন ধরনের গণনা GPU-র জন্য বিশেষভাবে উপযোগী?
43. "CPU দিয়ে Deep Learning চালানো যায় না"—এই বক্তব্যটি কেন ভুল?

Chapter 16

NVIDIA, CUDA এবং GPU Computing

Learning Objectives

- NVIDIA ও CUDA-এর ভূমিকা ব্যাখ্যা করতে পারা
- AMD ও ROCm-এর ধারণাগত পরিচিতি জানা
- NVIDIA কেন Deep Learning-এ প্রভাবশালী হয়ে উঠল তা বোঝা

মূল ধারণা (Main Concept)

১৬.১–১৬.২ NVIDIA ও CUDA

NVIDIA একটি প্রতিষ্ঠান যারা GPU তৈরি করে। **CUDA (Compute Unified Device Architecture)** হলো NVIDIA-র তৈরি একটি software platform, যা developer-দের NVIDIA GPU-তে সাধারণ গণনামূলক (general-purpose) কাজ চালানোর সুযোগ দেয়—শুধু graphics নয়।

১৬.৩ CUDA বনাম GPU

GPU হলো হার্ডওয়্যার, আর CUDA হলো সেই সফটওয়্যার স্তর যা developer-দের এই হার্ডওয়্যারের ক্ষমতা কার্যকরভাবে ব্যবহার করতে সাহায্য করে। CUDA ছাড়া GPU-র সমান্তরাল ক্ষমতা ব্যবহার করা প্রোগ্রামারদের জন্য অনেক বেশি কঠিন হতো।

১৬.৪ cuDNN

cuDNN হলো NVIDIA-র তৈরি একটি লাইব্রেরি, যা বিশেষভাবে Deep Neural Network-এর সাধারণ গণনা (যেমন convolution) দ্রুত সম্পন্ন করার জন্য অপ্টিমাইজ করা। এটি ধারণাগতভাবে CUDA-র উপর নির্মিত একটি অতিরিক্ত স্তর।

১৬.৫ কেন NVIDIA প্রভাবশালী হয়ে উঠল

বহু বছর ধরে CUDA-কে ঘিরে গড়ে ওঠা শক্তিশালী software ecosystem, ব্যাপক framework support (TensorFlow, PyTorch), এবং research community-র বহুল ব্যবহারের ফলে NVIDIA Deep Learning-এর জন্য হার্ডওয়্যার নির্বাচনে প্রভাবশালী অবস্থানে পৌঁছেছে।

১৬.৬–১৬.৭ AMD ও ROCm

AMD GPU তৈরিকারী আরেকটি বড় প্রতিষ্ঠান। AMD-র নিজস্ব GPU-computing software stack-কে বলা হয় **ROCm (Radeon Open Compute)**।

এখানে একটি গুরুত্বপূর্ণ স্পষ্টীকরণ প্রয়োজন: AMD GPU দিয়েও Deep Learning সম্ভব। NVIDIA ও AMD-র মধ্যে মূল পার্থক্য মূলত ecosystem-এর পরিপক্বতা (maturity), software support, লাইব্রেরির

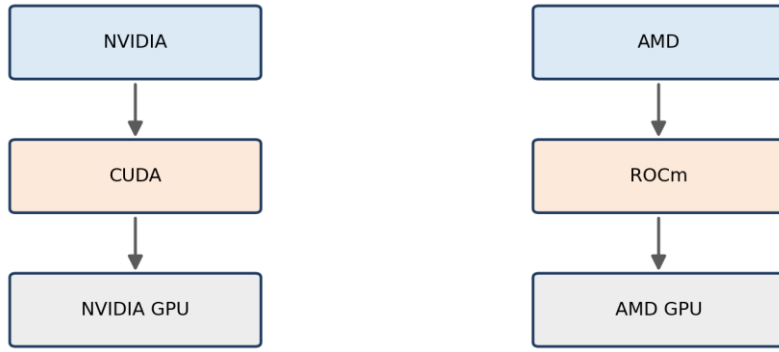
প্রাপ্যতা, framework-এর সাথে সংহতকরণ (integration), এবং ঐতিহাসিক গ্রহণযোগ্যতার (historical adoption) মধ্যে সীমাবদ্ধ—হার্ডওয়্যারের মৌলিক সক্ষমতার মধ্যে নয়।

Key Terms

NVIDIA • CUDA • cuDNN • AMD • ROCm

সারণি ১৬.১: NVIDIA CUDA বনাম AMD ROCm

বৈশিষ্ট্য	NVIDIA + CUDA	AMD + ROCm
মূল সক্ষমতা	Deep Learning সম্পাদনে সক্ষম	Deep Learning সম্পাদনে সক্ষম
Ecosystem পরিপক্বতা	দীর্ঘদিনের প্রতিষ্ঠিত ও ব্যাপক	ক্রমবর্ধমান, তুলনামূলক নবীন
Framework Support	ব্যাপক ও গভীরভাবে সংহত	উন্নতিশীল



চিত্র ১৬.১: NVIDIA → CUDA → GPU এবং AMD → ROCm → GPU-এর সম্পর্ক

Key Takeaways

- CUDA হলো NVIDIA-র সফটওয়্যার platform, যা GPU-তে general-purpose গণনা চালাতে সাহায্য করে।
- cuDNN হলো Deep Learning-নির্দিষ্ট গণনার জন্য optimize করা একটি লাইব্রেরি।
- AMD GPU দিয়েও Deep Learning সম্ভব; ROCm হলো এর সমতুল্য software stack।
- NVIDIA ও AMD-র পার্থক্য মূলত ecosystem ও software support-এর মধ্যে, হার্ডওয়্যারের মৌলিক সক্ষমতায় নয়।

Quick Review Questions

44. CUDA কী এবং এটি কেন প্রয়োজন?
45. "AMD GPU দিয়ে Deep Learning করা যায় না"—এই বক্তব্যটি কেন সঠিক নয়?
46. NVIDIA কেন Deep Learning ক্ষেত্রে প্রভাবশালী হয়ে উঠেছে?

Chapter 17

RAPIDS

Learning Objectives

- RAPIDS-এর মূল ধারণা সংক্ষেপে বোঝা
- RAPIDS ও CUDA-এর সম্পর্ক জানা

মূল ধারণা (Main Concept)

RAPIDS কী?

RAPIDS হলো NVIDIA-র একটি open-source সফটওয়্যার সংকলন (suite), যা GPU ব্যবহার করে **GPU-accelerated Data Science** সম্পন্ন করার সুযোগ দেয়। এর মধ্যে **cuDF** (GPU-ত্বরিত dataframe প্রক্রিয়াকরণ, pandas-এর অনুরূপ) এবং **cuML** (GPU-ত্বরিত traditional Machine Learning algorithm) অন্তর্ভুক্ত।

RAPIDS মূলত CUDA-এর উপর ভিত্তি করে নির্মিত, যা data science-এর সাধারণ কাজগুলো (data loading, cleaning, traditional ML) GPU-তে দ্রুততর করে তোলে।

স্পষ্টীকরণ

এটি স্পষ্ট করা প্রয়োজন যে RAPIDS নিজে Deep Learning-এর মূল কাঠামো নয়; এটি মূলত traditional data science ও ML workflow-কে GPU-তে দ্রুততর করার একটি টুলসেট।

Key Terms

RAPIDS • cuDF • cuML

Key Takeaways

- RAPIDS হলো GPU-accelerated Data Science-এর একটি সফটওয়্যার সংকলন।
- cuDF ও cuML যথাক্রমে dataframe প্রক্রিয়াকরণ ও traditional ML-কে GPU-তে দ্রুততর করে।
- RAPIDS Deep Learning-এর মূল কাঠামো নয়; এটি CUDA-এর উপর নির্মিত একটি সহায়ক টুলসেট।

Quick Review Questions

47. RAPIDS মূলত কী কাজে ব্যবহৃত হয়?
48. RAPIDS কি Deep Learning-এর মূল কাঠামো? ব্যাখ্যা করুন।

PART VIII

কেন ২০১২ একটি টার্নিং পয়েন্ট ছিল

Chapter 18

Deep Learning বিপ্লবের পূর্বে

Learning Objectives

- Deep Learning-এর প্রাথমিক সীমাবদ্ধতাগুলো চিহ্নিত করতে পারা
- কীভাবে এই বাধাগুলো ধীরে ধীরে দূর হলো তা বোঝা

মূল ধারণা (Main Concept)

১৮.১ পর্যাপ্ত Large Labeled Data-র অভাব

Neural Network-এর মৌলিক ধারণা বহু দশক ধরে পরিচিত থাকলেও, উপযুক্ত পরিমাণে বড় ও সঠিকভাবে label করা dataset-এর অভাবে এই মডেলগুলোর প্রকৃত সম্ভাবনা যাচাই করা সম্ভব হচ্ছিল না।

১৮.২ সীমিত Compute Power

সে সময়ের হার্ডওয়্যার বড় Neural Network প্রশিক্ষণের জন্য প্রয়োজনীয় বিপুল গণনা সম্পন্ন করার মতো সক্ষম ছিল না। GPU-ভিত্তিক general-purpose computing তখনও ব্যাপকভাবে প্রচলিত হয়নি।

১৮.৩ পূর্ববর্তী Training/Architecture সীমাবদ্ধতা

গভীর network প্রশিক্ষণে vanishing gradient-এর মতো সমস্যা এবং তুলনামূলক সরল architecture-এর কারণে Deep Learning-এর কার্যকারিতা সীমিত ছিল।

বাধা দূর হওয়া

সময়ের সাথে সাথে এই তিনটি বাধা—data, compute, এবং architecture—ধীরে ধীরে দুর্বল হতে শুরু করে: ইন্টারনেটের বিস্তারের সাথে সাথে বিপুল পরিমাণ data সংগ্রহ সম্ভব হয়, GPU computing সহজলভ্য হয়ে ওঠে, এবং গবেষকরা আরও কার্যকর training কৌশল উদ্ভাবন করেন।

Key Terms

Large Labeled Data • Compute Power

Key Takeaways

- Deep Learning বিপ্লবের পূর্বে প্রধান তিনটি বাধা ছিল: পর্যাপ্ত labeled data-র অভাব, সীমিত compute power, এবং architecture/training-এর সীমাবদ্ধতা।
- সময়ের সাথে data সংগ্রহ, GPU computing এবং উন্নত training কৌশলের মাধ্যমে এই বাধাগুলো ধীরে ধীরে দূর হয়।

Quick Review Questions

49. Deep Learning বিপ্লবের পূর্বে প্রধান তিনটি বাধা কী ছিল?

50. এই বাধাগুলো কীভাবে ধীরে ধীরে দূর হলো?

Chapter 19

Fei-Fei Li এবং ImageNet

Learning Objectives

- ImageNet dataset-এর গুরুত্ব ব্যাখ্যা করতে পারা
- Fei-Fei Li-র ভূমিকা জানা
- Data Annotation/Labeling-এর ধারণা বোঝা

মূল ধারণা (Main Concept)

ImageNet ও Fei-Fei Li

Fei-Fei Li একজন কম্পিউটার বিজ্ঞানী, যিনি **ImageNet** নামক একটি বৃহৎ, label করা image dataset তৈরির নেতৃত্ব দেন। ImageNet-এ লক্ষ লক্ষ ছবিকে হাজার হাজার category-তে ভাগ করে label করা হয়েছিল।

ImageNet-এর গুরুত্ব ছিল এই যে, এটি গবেষকদের প্রথমবারের মতো এত বিশাল পরিমাণে বৈচিত্র্যময় ও সঠিকভাবে label করা visual data দিয়ে বড় Neural Network প্রশিক্ষণের সুযোগ করে দেয়। পর্যাপ্ত labeled data ছাড়া বড় Deep Learning model-এর প্রকৃত সম্ভাবনা প্রমাণ করা সম্ভব ছিল না।

Data Annotation / Data Labeling

Data Annotation বা **Data Labeling** বলতে বোঝায় raw data-এর সাথে সঠিক তথ্য যুক্ত করার প্রক্রিয়া, যাতে model তা থেকে শিখতে পারে। উদাহরণস্বরূপ: একটি ছবিতে 'Cat' বা 'Dog' লেখা label যুক্ত করা, বস্তুর চারপাশে একটি **Bounding Box** আঁকা, ছবির প্রতিটি pixel-কে শ্রেণীবদ্ধ করে **Segmentation** করা, টেক্সট data-তে **Text Label** যুক্ত করা, অথবা অডিও data-কে টেক্সটে রূপান্তরিত করে **Audio Transcription** তৈরি করা।

Key Terms

Fei-Fei Li • ImageNet • Data Annotation • Data Labeling • Bounding Box • Segmentation

Key Takeaways

- Fei-Fei Li-র নেতৃত্বে তৈরি ImageNet ছিল একটি যুগান্তকারী, বৃহৎ ও সঠিকভাবে label করা image dataset।
- পর্যাপ্ত labeled data ছাড়া বড় Deep Learning model-এর সম্ভাবনা প্রমাণ করা সম্ভব ছিল না।
- Data Annotation বিভিন্ন রূপে হতে পারে—label, bounding box, segmentation, text label, বা audio transcription।

Quick Review Questions

51. ImageNet কেন গুরুত্বপূর্ণ ছিল?

52. Data Annotation-এর বিভিন্ন রূপের উদাহরণ দিন।

Chapter 20

ILSVRC

Learning Objectives

- ILSVRC-এর উদ্দেশ্য ব্যাখ্যা করতে পারা
- Computer Vision-এ এর গুরুত্ব বোঝা

মূল ধারণা (Main Concept)

ILSVRC কী?

ILSVRC (ImageNet Large Scale Visual Recognition Challenge) ছিল ImageNet dataset-এর একটি অংশ ব্যবহার করে আয়োজিত একটি বার্ষিক প্রতিযোগিতা, যেখানে বিভিন্ন গবেষক দল image classification-এর জন্য তাদের model-এর কার্যকারিতা পরীক্ষা করত।

ILSVRC একটি সাধারণ **Benchmark** হিসেবে কাজ করত, যা বিভিন্ন গবেষণা দলের ফলাফল সরাসরি তুলনা করার সুযোগ দিত। এটি Computer Vision গবেষণার অগ্রগতি ত্বরান্বিত করার ক্ষেত্রে গুরুত্বপূর্ণ ভূমিকা রাখে, কারণ এটি গবেষকদের একটি সাধারণ লক্ষ্যে কাজ করার সুযোগ তৈরি করে দেয়।

Key Terms

ILSVRC • Benchmark

Key Takeaways

- ILSVRC ছিল ImageNet-ভিত্তিক একটি বার্ষিক image classification প্রতিযোগিতা।
- এটি একটি সাধারণ benchmark হিসেবে কাজ করে Computer Vision গবেষণার তুলনা ও অগ্রগতি সহজ করে।

Quick Review Questions

53. ILSVRC কী এবং এটি কেন গুরুত্বপূর্ণ ছিল?

Chapter 21

২০১২ ILSVRC এবং AlexNet

Learning Objectives

- AlexNet-এর মূল অবদান ব্যাখ্যা করতে পারা
- ২০১২ সালের ফলাফলের ঐতিহাসিক গুরুত্ব বিশ্লেষণ করতে পারা
- AlexNet-এর সাথে যুক্ত গবেষকদের নাম জানা

মূল ধারণা (Main Concept)

২১.১–২১.৪ AlexNet ও এর নির্মাতারা

AlexNet ছিল একটি গভীর **Convolutional Neural Network (Deep CNN)**, যা **Alex Krizhevsky** তৈরি করেন তার সহকর্মী **Ilya Sutskever** এবং তত্ত্বাবধায়ক **Geoffrey Hinton**-এর সাথে মিলে। এই মডেলটি ২০১২ সালের ILSVRC প্রতিযোগিতায় জমা দেওয়া হয়।

২১.৫–২১.৬ Deep CNN ও GPU Acceleration

AlexNet একটি গভীর CNN architecture ব্যবহার করেছিল, যা একাধিক convolutional layer এবং fully-connected layer নিয়ে গঠিত ছিল। এই বিশাল model প্রশিক্ষণ সম্ভব হয়েছিল মূলত GPU acceleration-এর কারণে, যা সে সময়ের জন্য একটি উল্লেখযোগ্য প্রযুক্তিগত পদক্ষেপ ছিল।

২১.৭–২১.৯ ImageNet, ২০১২ ফলাফল ও ঐতিহাসিক গুরুত্ব

AlexNet ImageNet dataset-এর উপর প্রশিক্ষিত হয়েছিল এবং ২০১২ সালের ILSVRC প্রতিযোগিতায় প্রায় **১৫.৩%** top-5 error rate অর্জন করে, যেখানে পূর্ববর্তী সেরা ফলাফল ছিল প্রায় **২৬.২%**। এই বিশাল ব্যবধান সে সময়ের গবেষণা সম্প্রদায়কে বিম্বিত করেছিল এবং বড় আকারে Deep Learning-এর শক্তি স্পষ্টভাবে প্রদর্শন করেছিল।

এটি স্পষ্ট করা প্রয়োজন যে AlexNet এককভাবে Deep Learning "আবিষ্কার" করেনি—Neural Network-এর মৌলিক ধারণা এর অনেক আগে থেকেই বিদ্যমান ছিল। তবে AlexNet-এর এই ফলাফল আধুনিক Deep Learning বিপ্লবের একটি প্রধান অনুঘটক (catalyst) হিসেবে কাজ করে, যা গবেষণা ও শিল্পক্ষেত্রে Deep Learning-এর প্রতি আগ্রহ ব্যাপকভাবে বাড়িয়ে দেয়।

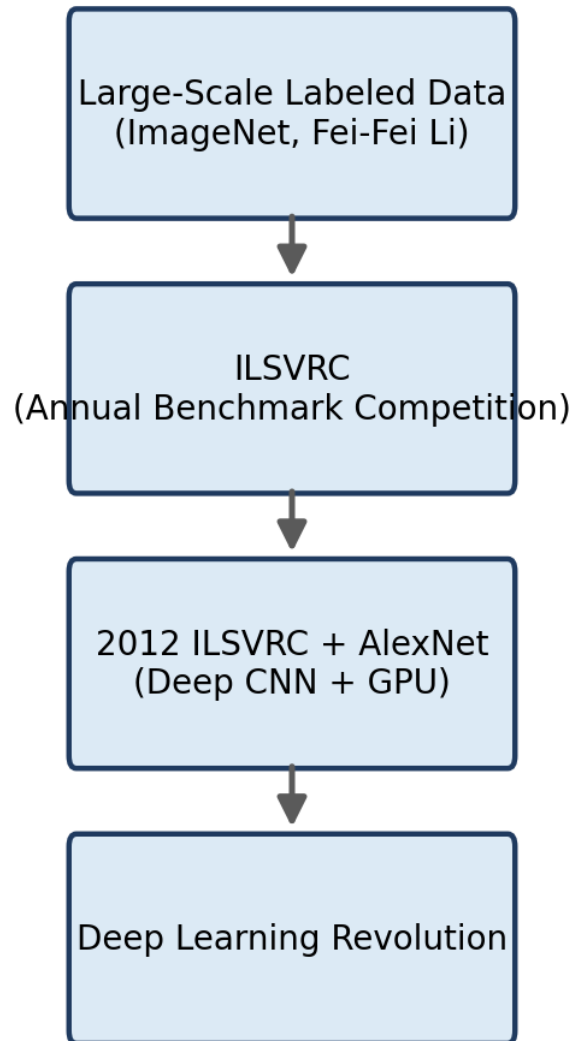
Key Terms

AlexNet • Alex Krizhevsky • Ilya Sutskever • Geoffrey Hinton • Deep CNN • GPU Acceleration

সারণি ২১.১: ILSVRC ২০১২ ফলাফল সংক্ষেপ

মডেল	Top-5 Error Rate
AlexNet (বিজয়ী)	প্রায় ১৫.৩%

মডেল	Top-5 Error Rate
পূর্ববর্তী সেরা পদ্ধতি	প্রায় ২৬.২%



চিত্র ২১.১: ImageNet → ILSVRC → AlexNet → Deep Learning বিপ্লব

Key Takeaways

- AlexNet ছিল Alex Krizhevsky, Ilya Sutskever ও Geoffrey Hinton-এর তৈরি একটি গভীর CNN।
- ২০১২ সালে AlexNet প্রায় ১৫.৩% top-5 error rate অর্জন করে, যেখানে পূর্ববর্তী সেরা ছিল প্রায় ২৬.২%।
- এই ফলাফল সম্ভব হয়েছিল বড় labeled dataset (ImageNet) ও GPU acceleration-এর সমন্বয়ে।
- AlexNet Deep Learning আবিষ্কার করেনি, কিন্তু এটি আধুনিক Deep Learning বিপ্লবের একটি প্রধান অনুঘটক ছিল।

Quick Review Questions

54. AlexNet-এর সাফল্যের পেছনে কোন কোন উপাদান ভূমিকা রেখেছিল?
55. ২০১২ সালের ফলাফল কেন ঐতিহাসিকভাবে গুরুত্বপূর্ণ?
56. "AlexNet Deep Learning আবিষ্কার করেছিল"—এই বক্তব্যটি কেন সম্পূর্ণ সঠিক নয়?

PART IX

নিউরাল নেটওয়ার্ক আর্কিটেকচারের বিবর্তন

Chapter 22

ANN / MLP

Learning Objectives

- ANN ও MLP-এর সাধারণ ধারণা বোঝা
- Dense/Fully Connected layer-এর অর্থ জানা

মূল ধারণা (Main Concept)

সাধারণ-উদ্দেশ্য Architecture

ANN (Artificial Neural Network) এবং **MLP (Multi-Layer Perceptron)** পরিভাষা দুটি প্রায়ই একে অপরের সমার্থক হিসেবে ব্যবহৃত হয়, যা পূর্ববর্তী অধ্যায়গুলোতে আলোচিত Input, Hidden ও Output Layer সমৃদ্ধ সাধারণ Neural Network-কে বোঝায়।

এই ধরনের network-এ সাধারণত **Dense** বা **Fully Connected** layer ব্যবহৃত হয়, যেখানে একটি layer-এর প্রতিটি neuron পরবর্তী layer-এর প্রতিটি neuron-এর সাথে সংযুক্ত থাকে। ANN/MLP একটি সাধারণ-উদ্দেশ্য (general-purpose) architecture, যা বিভিন্ন ধরনের সমস্যায় ব্যবহার করা যায়, তবে ছবি বা sequence-এর মতো বিশেষায়িত data-র জন্য পরবর্তীতে আরও উপযোগী architecture (CNN, RNN) তৈরি হয়েছে।

Key Terms

ANN • MLP • Dense Layer • Fully Connected Layer

Key Takeaways

- ANN ও MLP মূলত একই ধরনের সাধারণ-উদ্দেশ্য Neural Network-কে নির্দেশ করে।
- Dense/Fully Connected layer-এ একটি layer-এর প্রতিটি neuron পরবর্তী layer-এর প্রতিটি neuron-এর সাথে সংযুক্ত থাকে।

Quick Review Questions

57. ANN ও MLP-এর মধ্যে সম্পর্ক কী?
58. Fully Connected layer বলতে কী বোঝায়?

Chapter 23

CNN

Learning Objectives

- CNN কেন তৈরি হয়েছিল তা ব্যাখ্যা করতে পারা
- Convolution, Kernel/Filter ও Feature Map-এর ধারণা বোঝা
- Hierarchical Feature Learning-এর ধারণা ব্যাখ্যা করতে পারা

মূল ধারণা (Main Concept)

CNN কেন তৈরি হয়েছিল

CNN (Convolutional Neural Network) বিশেষভাবে ছবির মতো data প্রক্রিয়াকরণের জন্য তৈরি করা হয়েছে, যেখানে স্থানিক (spatial) স্থানীয় প্যাটার্ন (local patterns)—যেমন edge, texture—গুরুত্বপূর্ণ। সাধারণ Fully Connected network ছবির প্রতিটি pixel-কে আলাদাভাবে বিবেচনা করে, যা স্থানিক গঠন উপেক্ষা করে এবং অত্যন্ত বেশি parameter দাবি করে।

Convolution, Kernel/Filter ও Feature Map

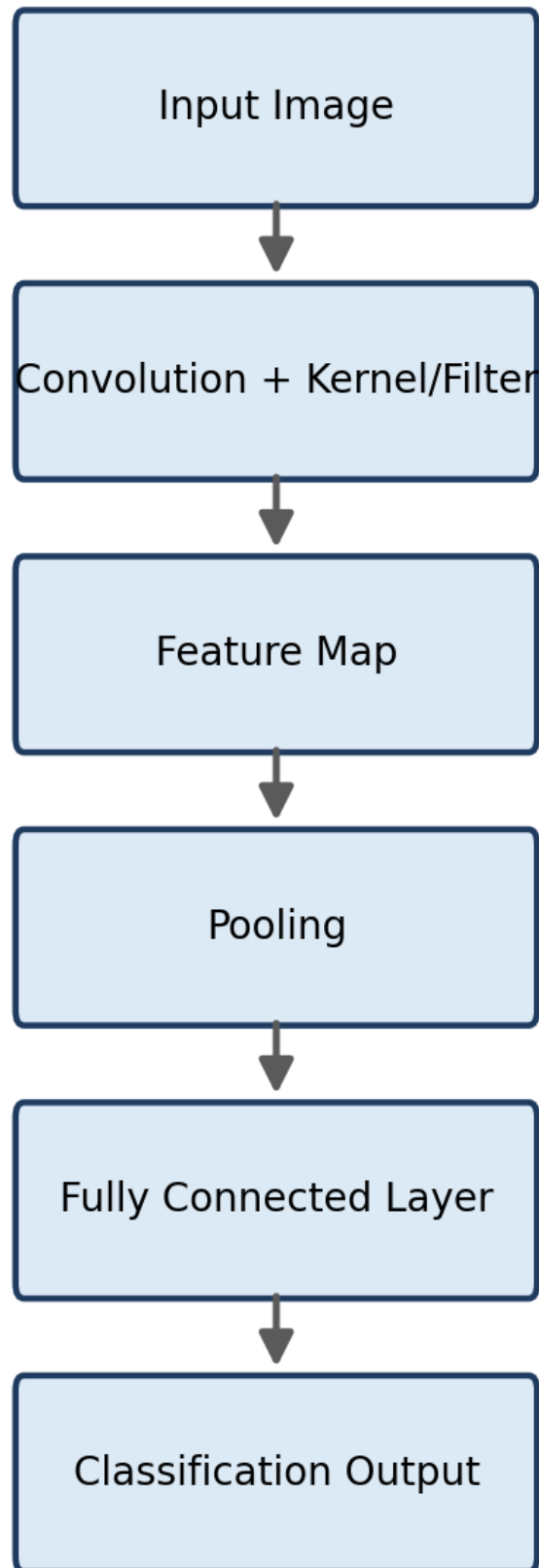
Convolution হলো একটি গাণিতিক প্রক্রিয়া, যেখানে একটি ছোট **Kernel** বা **Filter** ছবির উপর দিয়ে ধীরে ধীরে সরে গিয়ে স্থানীয় প্যাটার্ন শনাক্ত করে। এই প্রক্রিয়ার ফলাফলকে **Feature Map** বলা হয়, যা ছবির নির্দিষ্ট বৈশিষ্ট্যের উপস্থিতি নির্দেশ করে।

Hierarchical Feature Learning

CNN-এর একাধিক স্তর ধাপে ধাপে ক্রমশ জটিল প্যাটার্ন শেখে: প্রাথমিক স্তর edge শনাক্ত করে, পরবর্তী স্তর texture ও shape শনাক্ত করে, তারপর object part, এবং সবশেষে উচ্চ-স্তরের feature ব্যবহার করে classification সম্পন্ন করা হয়।

Key Terms

CNN • Convolution • Kernel • Filter • Feature Map • Hierarchical Feature Learning



চিত্র ২৩.১: CNN-এর মাধ্যমে ধাপে ধাপে জটিলতর feature শেখা

Key Takeaways

- CNN বিশেষভাবে ছবির মতো স্থানিক গঠনসম্পন্ন data-র জন্য উপযোগী।
- Convolution প্রক্রিয়ায় Kernel/Filter ব্যবহার করে Feature Map তৈরি করা হয়।

- CNN স্তরভিত্তিকভাবে সরল থেকে জটিল প্যাটার্ন শেখে (edge → texture → object part → high-level feature)।

Quick Review Questions

59. CNN কেন সাধারণ Fully Connected network-এর চেয়ে ছবির জন্য বেশি উপযোগী?
60. Kernel/Filter ও Feature Map-এর সম্পর্ক ব্যাখ্যা করুন।

Chapter 24

RNN, LSTM এবং GRU

Learning Objectives

- RNN কীভাবে sequential data প্রক্রিয়া করে তা বোঝা
- LSTM ও GRU-এর উদ্দেশ্য ও সুবিধা ধারণাগতভাবে ব্যাখ্যা করতে পারা

মূল ধারণা (Main Concept)

RNN ও Sequential Data

RNN (Recurrent Neural Network) বিশেষভাবে **Sequential Data**—যেমন টেক্সট, Time Series, অডিও, বা signal data—প্রক্রিয়াকরণের জন্য তৈরি করা হয়েছে, যেখানে ক্রম (order) গুরুত্বপূর্ণ। RNN একটি **Hidden State** বজায় রাখে, যা পূর্ববর্তী ধাপের তথ্য মনে রাখে এবং বর্তমান ধাপের সিদ্ধান্তে ব্যবহার করে।

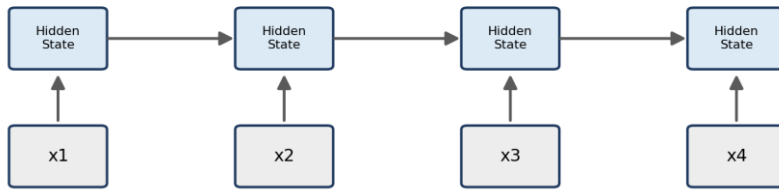
LSTM ও GRU

সাধারণ RNN দীর্ঘ sequence-এ পুরোনো তথ্য মনে রাখতে সমস্যায় পড়ে। এই সীমাবদ্ধতা কাটিয়ে ওঠার জন্য **LSTM (Long Short-Term Memory)** এবং **GRU (Gated Recurrent Unit)** তৈরি করা হয়েছে, যেগুলো বিশেষ gate ব্যবহার করে গুরুত্বপূর্ণ তথ্য দীর্ঘ সময় ধরে মনে রাখতে এবং অপ্রয়োজনীয় তথ্য ভুলে যেতে সক্ষম। এই বইয়ে এদের অভ্যন্তরীণ gate-এর গাণিতিক বিস্তারিত আলোচনা করা হয়নি; শুধু ধারণাগত উদ্দেশ্য ব্যাখ্যা করা হয়েছে।

Key Terms

RNN • Sequential Data • Hidden State • LSTM • GRU

Previous hidden state carries information forward through the sequence



চিত্র ২৪.১: RNN-এ sequence বরাবর hidden state প্রবাহিত হওয়া

Key Takeaways

- RNN sequential data প্রক্রিয়াকরণের জন্য উপযোগী, কারণ এটি Hidden State-এর মাধ্যমে পূর্ববর্তী তথ্য মনে রাখে।
- LSTM ও GRU সাধারণ RNN-এর দীর্ঘ sequence-এ তথ্য মনে রাখার সীমাবদ্ধতা কাটিয়ে ওঠার জন্য তৈরি করা হয়েছে।

Quick Review Questions

61. RNN কীভাবে sequential data প্রক্রিয়া করে?
62. LSTM/GRU সাধারণ RNN-এর তুলনায় কী সুবিধা প্রদান করে?

Chapter 25

Attention এবং Transformer

Learning Objectives

- Attention-এর প্রয়োজনীয়তা ব্যাখ্যা করতে পারা
- Self-Attention-এর মূল ধারণা বোঝা
- Transformer কেন গুরুত্বপূর্ণ হয়ে উঠল তা ব্যাখ্যা করতে পারা

মূল ধারণা (Main Concept)

২৫.১–২৫.২ কেন Attention প্রয়োজন

RNN-ভিত্তিক model দীর্ঘ sequence-এ দূরবর্তী শব্দ বা উপাদানের মধ্যে সম্পর্ক ধরে রাখতে অসুবিধায় পড়ে। **Attention** পদ্ধতি model-কে সরাসরি sequence-এর যেকোনো অংশের দিকে 'মনোযোগ' দেওয়ার সুযোগ দেয়, দূরত্ব নির্বিশেষে।

২৫.৩ Self-Attention

Self-Attention-এ sequence-এর প্রতিটি উপাদান একই sequence-এর অন্য সব উপাদানের সাথে তার সম্পর্ক নির্ধারণ করে। এর ফলে model বুঝতে পারে কোন শব্দ কোন শব্দের সাথে প্রাসঙ্গিকভাবে যুক্ত, দূরত্ব নির্বিশেষে।

২৫.৪–২৫.৫ Transformer ও এর গুরুত্ব

Transformer হলো একটি architecture, যা সম্পূর্ণভাবে Self-Attention-এর উপর ভিত্তি করে তৈরি, RNN-এর ধারাবাহিক (sequential) গণনার প্রয়োজনীয়তা ছাড়াই। এর ফলে Transformer একই সাথে সম্পূর্ণ sequence প্রক্রিয়া করতে পারে, যা প্রশিক্ষণকে দ্রুততর ও বেশি parallelizable করে তোলে।

Transformer-এর এই দক্ষতা ও কার্যকারিতা এটিকে আধুনিক Natural Language Processing এবং পরবর্তীতে Large Language Model-এর মূল ভিত্তি হিসেবে প্রতিষ্ঠিত করেছে।

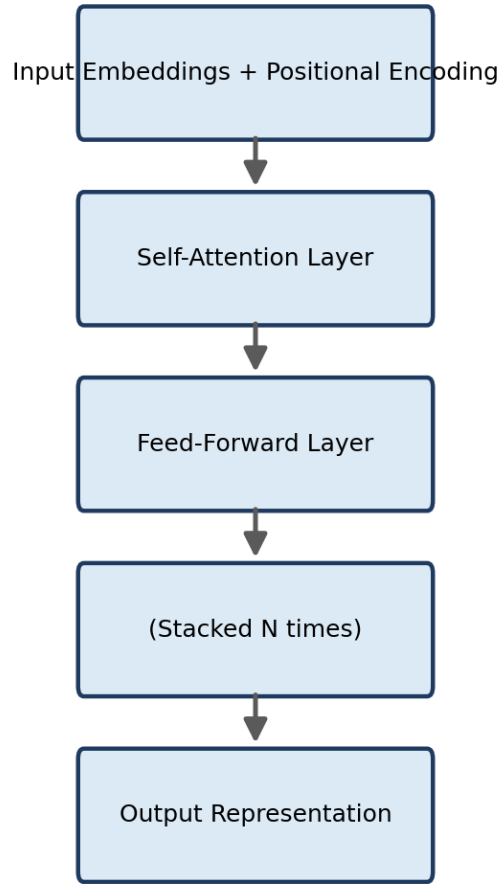
Key Terms

Attention • Self-Attention • Transformer

Self-Attention: each word relates to every other word in the sequence



চিত্র ২৫.১: Self-Attention — sequence-এর প্রতিটি অংশ অন্য সব অংশের সাথে সম্পর্কিত



চিত্র ২৫.২: একটি সরলীকৃত Transformer Block

Key Takeaways

- Attention model-কে sequence-এর যেকোনো অংশে সরাসরি মনোযোগ দেওয়ার সুযোগ দেয়, দূরত্ব নির্বিশেষে।
- Self-Attention-এ sequence-এর প্রতিটি উপাদান অন্য সব উপাদানের সাথে সম্পর্ক নির্ধারণ করে।
- Transformer সম্পূর্ণভাবে Attention-ভিত্তিক, যা একে দ্রুত ও parallelizable করে তোলে এবং আধুনিক LLM-এর ভিত্তি হিসেবে কাজ করে।

Quick Review Questions

63. RNN-এর তুলনায় Attention কী সুবিধা প্রদান করে?
64. Self-Attention কী বোঝায়?
65. Transformer কেন আধুনিক NLP-তে এত গুরুত্বপূর্ণ হয়ে উঠেছে?

PART X

LLM এবং আধুনিক Generative AI

Chapter 26

Large Language Model

LLM (Large Language Model)

LLM হলো একটি বিশাল, সাধারণত Transformer-ভিত্তিক Neural Network, যা বিপুল পরিমাণ টেক্সট data থেকে ভাষার প্যাটার্ন শিখে টেক্সট বোঝা ও তৈরি করতে সক্ষম।

Learning Objectives

- LLM কী তা ব্যাখ্যা করতে পারা
- Token, Tokenization ও Context-এর ধারণা বোঝা
- Next-token Prediction-এর ভিত্তিতে LLM কীভাবে টেক্সট তৈরি করে তা ব্যাখ্যা করতে পারা

মূল ধারণা (Main Concept)

LLM কী?

LLM (Large Language Model) হলো একটি বিশাল, সাধারণত Transformer architecture-ভিত্তিক Neural Network, যা বিপুল পরিমাণ টেক্সট data থেকে ভাষার প্যাটার্ন শিখে টেক্সট বোঝা ও তৈরি করতে সক্ষম। এর 'Large' নামকরণ এসেছে এর বিশাল সংখ্যক Parameters এবং প্রশিক্ষণ data-র আকার থেকে।

Token, Tokenization ও Context

LLM সরাসরি শব্দ বা বাক্য প্রক্রিয়া করে না; বরং টেক্সটকে ছোট ছোট একক—**Token**-এ ভাগ করে, যাকে **Tokenization** বলা হয়। একটি token একটি সম্পূর্ণ শব্দ, শব্দের অংশ, বা একটি চিহ্নও হতে পারে। **Context** বলতে বোঝায় model বর্তমান সময়ে কতটা পূর্ববর্তী টেক্সট বিবেচনা করতে পারে।

Training ও Inference

Training পর্যায়ে model বিপুল পরিমাণ টেক্সট data থেকে ভাষার প্যাটার্ন শেখে, এবং **Inference** পর্যায়ে সেই শেখা জ্ঞান ব্যবহার করে নতুন input-এর জন্য output তৈরি করে।

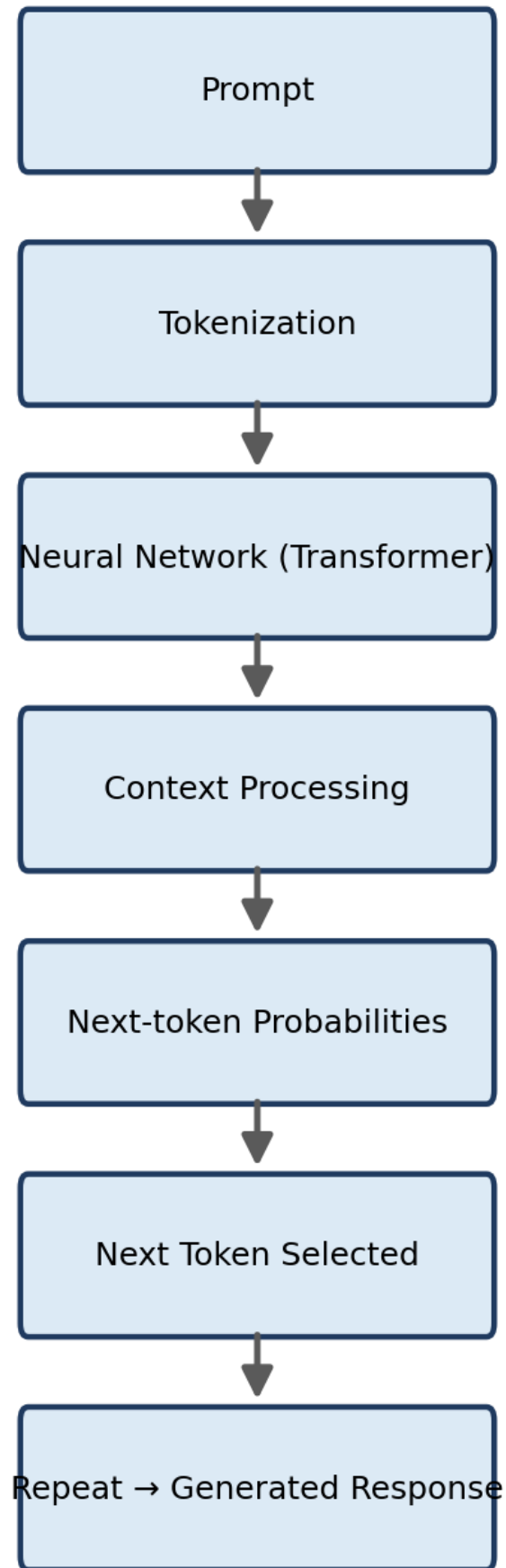
Next-token Prediction ও Generation Process

LLM মূলত **Next-token Prediction**-এর মাধ্যমে কাজ করে—অর্থাৎ, এখন পর্যন্ত দেওয়া টেক্সটের ভিত্তিতে পরবর্তী সবচেয়ে সম্ভাব্য token অনুমান করে। এই প্রক্রিয়াটি নিম্নরূপ: একটি Prompt দেওয়া হয়, তা Tokenization-এর মাধ্যমে token-এ ভাগ হয়, Neural Network (Transformer) সেই token-গুলো প্রক্রিয়া করে Context বিশ্লেষণ করে, প্রতিটি সম্ভাব্য পরবর্তী token-এর জন্য একটি সম্ভাব্যতা

(probability) নির্ধারণ করে, একটি token নির্বাচন করা হয়, এবং এই সম্পূর্ণ প্রক্রিয়া পুনরাবৃত্তি করে ধীরে ধীরে একটি সম্পূর্ণ response তৈরি হয়।

Key Terms

LLM • Token • Tokenization • Context • Next-token Prediction • Training • Inference



চিত্র ২৬.১: LLM-এর টেক্সট তৈরি প্রক্রিয়া (Generation Pipeline)

Key Takeaways

- LLM হলো একটি বিশাল Transformer-ভিত্তিক Neural Network, যা বিপুল টেক্সট data থেকে ভাষার প্যাটার্ন শেখে।
- টেক্সটকে Token-এ ভাগ করার প্রক্রিয়াকে Tokenization বলা হয়।
- LLM মূলত Next-token Prediction-এর মাধ্যমে ধাপে ধাপে টেক্সট তৈরি করে।

Quick Review Questions

66. Token ও Tokenization বলতে কী বোঝায়?
67. LLM কীভাবে Next-token Prediction ব্যবহার করে টেক্সট তৈরি করে, ধাপে ধাপে ব্যাখ্যা করুন।
68. Training ও Inference-এর মধ্যে পার্থক্য কী?

Chapter 27

ChatGPT এবং Gemini

Learning Objectives

- ChatGPT ও Gemini কী তা ধারণাগতভাবে বোঝা
- Product/System বনাম Underlying Model-এর পার্থক্য বুঝতে পারা
- Multimodal ক্ষমতার ধারণাগত পরিচিতি জানা

মূল ধারণা (Main Concept)

ChatGPT ও Gemini কী

ChatGPT (OpenAI দ্বারা তৈরি) এবং **Gemini** (Google DeepMind দ্বারা তৈরি) হলো জনপ্রিয় AI product/system, যেগুলো LLM প্রযুক্তির উপর ভিত্তি করে তৈরি। এই দুটোই ব্যবহারকারীদের সাথে কথোপকথনের মাধ্যমে প্রশ্নের উত্তর দেওয়া, লেখা তৈরি করা, এবং অন্যান্য ভাষাভিত্তিক কাজ সম্পন্ন করতে সক্ষম।

Product/System বনাম Underlying Model

এটি বোঝা গুরুত্বপূর্ণ যে ChatGPT বা Gemini শুধুমাত্র একটিমাত্র Neural Network নয়। এগুলো হলো সম্পূর্ণ Product বা System, যার মধ্যে অন্তর্নিহিত (underlying) LLM ছাড়াও অতিরিক্ত নিরাপত্তা ব্যবস্থা, ব্যবহারকারী interface, memory ব্যবস্থাপনা, এবং অন্যান্য সহায়ক প্রযুক্তিগত উপাদান অন্তর্ভুক্ত থাকে।

Multimodal ক্ষমতা

আধুনিক সিস্টেমগুলো প্রায়ই **Multimodal**—অর্থাৎ, শুধু টেক্সট নয়, ছবি, অডিও, বা অন্যান্য ধরনের data-ও বুঝতে ও তৈরি করতে সক্ষম। এই বইয়ে এই সিস্টেমগুলোর সঠিকভাবে নিশ্চিত না-হওয়া অভ্যন্তরীণ architecture, প্যারামিটার সংখ্যা, বা প্রশিক্ষণ dataset সম্পর্কে কোনো নির্দিষ্ট দাবি করা হয়নি, কারণ এই তথ্যগুলো সাধারণত প্রকাশ্যে নিশ্চিতভাবে জানা যায় না।

Key Terms

ChatGPT • Gemini • Multimodal

Key Takeaways

- ChatGPT ও Gemini হলো LLM-ভিত্তিক AI product/system, যথাক্রমে OpenAI ও Google DeepMind দ্বারা তৈরি।
- এই সিস্টেমগুলো শুধু একটি Neural Network নয়—এগুলোতে অতিরিক্ত নিরাপত্তা ব্যবস্থা ও প্রযুক্তিগত উপাদান অন্তর্ভুক্ত থাকে।
- Multimodal ক্ষমতা মানে টেক্সটের পাশাপাশি ছবি, অডিওর মতো বিভিন্ন ধরনের data প্রক্রিয়া করতে পারা।

Quick Review Questions

69. ChatGPT/Gemini-কে কেন শুধুমাত্র 'একটি Neural Network' বলা সঠিক নয়?
70. Multimodal ক্ষমতা বলতে কী বোঝায়?

PART XI

সম্পূর্ণ সংযোগ

Chapter 28

নিউরন থেকে আধুনিক AI পর্যন্ত

Learning Objectives

- বইয়ের সম্পূর্ণ ধারণাগত যাত্রা সংক্ষেপে পুনরালোচনা করতে পারা
- স্থাপত্যগত (architectural) এবং শেখার প্রক্রিয়াগত (learning mechanism) দুই ধারার সংযোগ বুঝতে পারা

মূল ধারণা (Main Concept)

সম্পূর্ণ যাত্রা

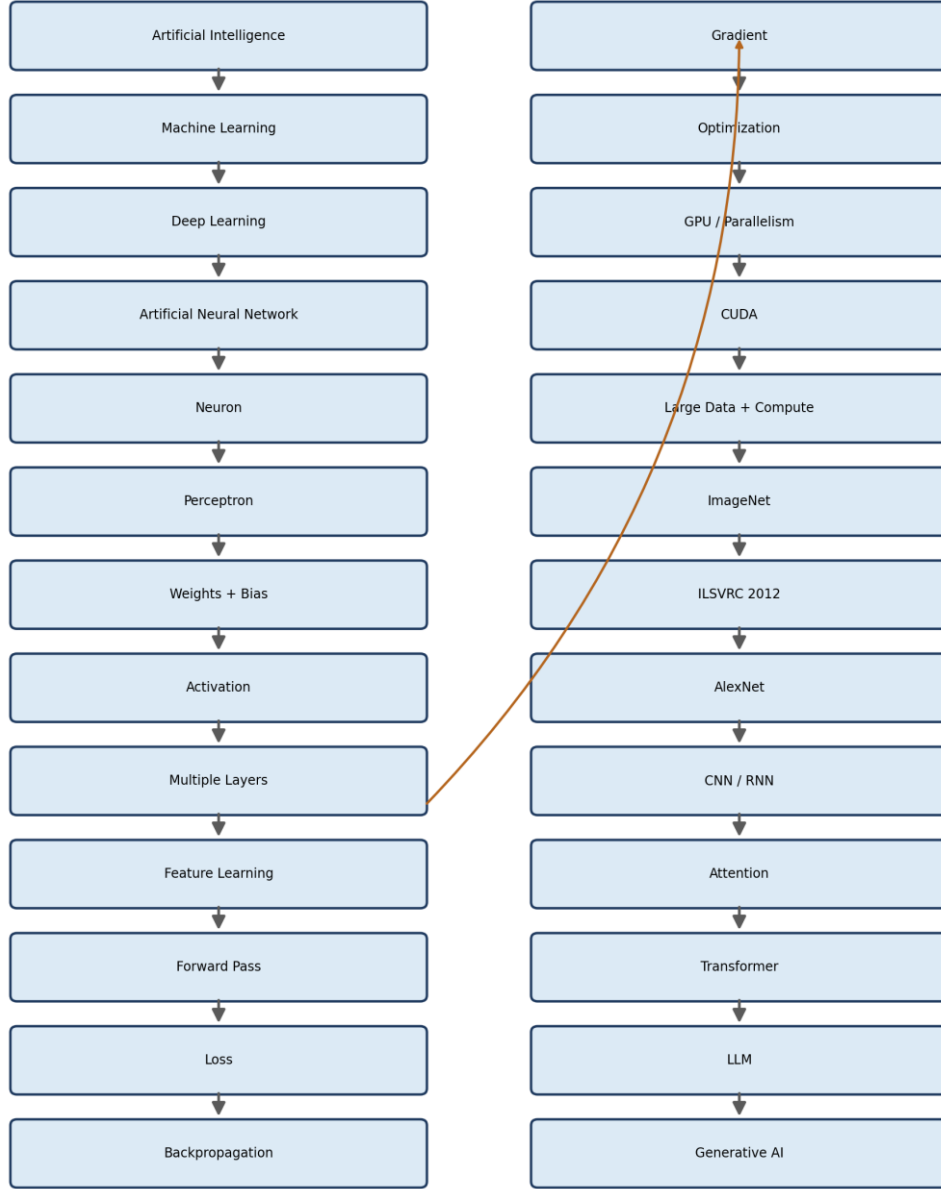
এই বইয়ে আমরা একটি দীর্ঘ ধারণাগত যাত্রা অতিক্রম করেছি—জৈবিক নিউরন থেকে শুরু করে আধুনিক Generative AI পর্যন্ত। প্রথমে আমরা দেখেছি কীভাবে একটি Biological Neuron একটি Artificial Neuron-এ রূপান্তরিত হয়, যা পরবর্তীতে Perceptron-এ বিকশিত হয়। Single-Layer Perceptron-এর সীমাবদ্ধতা (XOR Problem) আমাদের একাধিক Layer ব্যবহার করতে বাধ্য করে, যা থেকে MLP/ANN এবং পরবর্তীতে Deep Neural Network-এর জন্ম হয়।

Deep Neural Network Automatic Feature Learning-এর মাধ্যমে raw data থেকে নিজে থেকেই useful feature শিখতে সক্ষম হয়। এই ভিত্তির উপর দাঁড়িয়ে বিশেষায়িত architecture—ছবির জন্য CNN এবং sequence-এর জন্য RNN—তৈরি হয়। পরবর্তীতে Attention ও Transformer এই স্থাপত্যগুলোর সীমাবদ্ধতা দূর করে, যা আধুনিক LLM এবং সামগ্রিকভাবে Modern Generative AI-এর ভিত্তি স্থাপন করে।

শেখার প্রক্রিয়ার সংযোগ

এই স্থাপত্যগত যাত্রার পাশাপাশি, প্রতিটি architecture একটি অভিন্ন শেখার প্রক্রিয়া অনুসরণ করে: Input থেকে Forward Pass-এর মাধ্যমে Prediction তৈরি হয়, Prediction ও প্রকৃত লক্ষ্যের পার্থক্য থেকে Loss গণনা করা হয়, Backpropagation-এর মাধ্যমে Gradient নির্ণয় করা হয়, এবং Optimization-এর মাধ্যমে Weight ও Bias হালনাগাদ করা হয়—এই সম্পূর্ণ চক্র বারবার পুনরাবৃত্তি করে model ক্রমশ উন্নত হতে থাকে।

এই দুটি ধারা—architecture-এর বিবর্তন এবং শেখার প্রক্রিয়ার ধারাবাহিকতা—একসাথে মিলেই Deep Learning-এর সম্পূর্ণ চিত্র তৈরি করে। পরবর্তী অংশে (Historical Timeline ও Quick Revision Sheet) এই সম্পূর্ণ যাত্রার একটি সংক্ষিপ্ত ও দ্রুত পর্যালোচনাযোগ্য রূপ উপস্থাপন করা হয়েছে।



চিত্র ২৮.১: নিউরন থেকে আধুনিক Generative AI পর্যন্ত সম্পূর্ণ ধারণাগত সংযোগ

Key Takeaways

- Deep Learning-এর যাত্রা Biological Neuron থেকে শুরু হয়ে Perceptron, MLP, CNN/RNN, Attention, Transformer হয়ে আধুনিক LLM পর্যন্ত বিস্তৃত।
- প্রতিটি architecture-এর মূলে একটি অভিন্ন শেখার প্রক্রিয়া কাজ করে: Forward Pass → Loss → Backpropagation → Gradient → Optimization।
- Architecture-এর বিবর্তন এবং শেখার প্রক্রিয়া—এই দুই ধারা একসাথে মিলেই Deep Learning-এর সম্পূর্ণ চিত্র গঠন করে।

Quick Review Questions

71. Biological Neuron থেকে LLM পর্যন্ত সম্পূর্ণ ধারণাগত যাত্রাটি নিজের ভাষায় সংক্ষেপে বর্ণনা করুন।

72. প্রতিটি Neural Network architecture-এর মূলে থাকা অভিন্ন শেখার প্রক্রিয়াটি কী?

Historical Timeline



চিত্র: Deep Learning-এর ইতিহাসের সংক্ষিপ্ত সময়রেখা (ধারণাগত দিকনির্দেশনার জন্য)

১৯৪৩ — McCulloch-Pitts Artificial Neuron

Warren McCulloch ও Walter Pitts একটি গাণিতিক মডেল প্রস্তাব করেন, যা দেখায় কীভাবে সরল কৃত্রিম নিউরন যৌক্তিক (logical) গণনা সম্পন্ন করতে পারে। এটি Artificial Neural Network-এর ধারণার সূচনাবিন্দু হিসেবে বিবেচিত হয়।

১৯৫৮ — Perceptron

Frank Rosenblatt Perceptron মডেল উদ্ভাবন করেন, যা Binary Classification সম্পন্ন করতে সক্ষম একটি শেখার-উপযোগী (learnable) মডেল।

১৯৮৬ — Backpropagation জনপ্রিয় হয়

Rumelhart, Hinton ও Williams-এর গবেষণাপত্রের মাধ্যমে Backpropagation পদ্ধতি বহুস্তর Neural Network প্রশিক্ষণের একটি ব্যবহারিক পদ্ধতি হিসেবে ব্যাপক পরিচিতি লাভ করে।

AI Winter — সীমিত অর্থায়ন ও আগ্রহ

বিভিন্ন সময়ে অতিরঞ্জিত প্রত্যাশা ও সীমিত ফলাফলের কারণে AI গবেষণায় অর্থায়ন ও আগ্রহ উল্লেখযোগ্যভাবে হ্রাস পায়, যা 'AI Winter' নামে পরিচিত। এটি কোনো একক সময়কাল নয়, বরং একাধিক পর্যায়ে ঘটেছে।

ImageNet Era — বৃহৎ Labeled Visual Data

Fei-Fei Li-র নেতৃত্বে ImageNet তৈরির মাধ্যমে গবেষকরা প্রথমবারের মতো বিশাল পরিমাণে সঠিকভাবে label করা visual data ব্যবহারের সুযোগ পান।

২০১২ — AlexNet + ILSVRC

AlexNet ILSVRC প্রতিযোগিতায় বিশাল ব্যবধানে জয়লাভ করে (প্রায় ১৫.৩% বনাম ২৬.২% top-5 error), যা আধুনিক Deep Learning বিপ্লবের একটি প্রধান অনুঘটক হিসেবে কাজ করে।

২০১৭ — Transformer Architecture

"Attention Is All You Need" গবেষণাপত্রের মাধ্যমে Transformer architecture প্রবর্তিত হয়, যা পরবর্তীতে আধুনিক LLM-এর মূল ভিত্তি হয়ে ওঠে।

এই সময়রেখাটি শুধুমাত্র ধারণাগত দিকনির্দেশনার (conceptual orientation) জন্য তৈরি করা হয়েছে। এখানে উল্লেখিত ঘটনাগুলোই Deep Learning-এর ইতিহাসের একমাত্র গুরুত্বপূর্ণ ঘটনা—এমনটি ধরে নেওয়া ঠিক হবে না। এর মাঝেও আরও অনেক গুরুত্বপূর্ণ গবেষণা ও অবদান রয়েছে, যা এই সংক্ষিপ্ত পরিচিতিতে অন্তর্ভুক্ত করা হয়নি।

Final Summary

এই বইটি Artificial Intelligence, Machine Learning এবং Deep Learning-এর মধ্যকার সম্পর্ক দিয়ে শুরু হয়ে, জৈবিক নিউরন থেকে কৃত্রিম নিউরন, Perceptron, বহুস্তর নিউরাল নেটওয়ার্ক, শেখার প্রক্রিয়া (Forward Propagation, Loss, Backpropagation, Optimization), হার্ডওয়্যারের ভূমিকা, ২০১২ সালের ঐতিহাসিক টার্নিং পয়েন্ট, আধুনিক architecture-এর বিবর্তন (CNN, RNN, Attention, Transformer), এবং সবশেষে LLM ও Generative AI পর্যন্ত একটি সম্পূর্ণ ধারণাগত যাত্রা সম্পন্ন করেছে।

এই বইয়ের লক্ষ্য ছিল একটি শক্তিশালী conceptual foundation তৈরি করা, যা পাঠককে ভবিষ্যতে আরও গভীর গবেষণাধর্মী উপকরণে অগ্রসর হওয়ার জন্য প্রস্তুত করবে। পরবর্তী পৃষ্ঠাগুলোতে দ্রুত পুনরালোচনার জন্য একটি Glossary এবং একটি Quick Revision Sheet সংযুক্ত করা হয়েছে।

Glossary

Artificial Intelligence (AI)

কম্পিউটার সিস্টেমকে মানুষের মতো বুদ্ধিবৃত্তিক আচরণ প্রদর্শনের উপযোগী করে তোলার ক্ষেত্র।

Machine Learning (ML)

AI-এর একটি উপশাখা, যেখানে সিস্টেম data থেকে patterns শিখে সিদ্ধান্ত নেয়, প্রতিটি নিয়ম হাতে প্রোগ্রাম না করেই।

Deep Learning (DL)

ML-এর একটি শাখা, যেখানে বহুস্তরবিশিষ্ট Neural Network ব্যবহার করে data থেকে জটিল patterns শেখানো হয়।

Neural Network

একাধিক পরস্পর-সংযুক্ত neuron নিয়ে গঠিত একটি গণনামূলক মডেল।

Neuron

Neural Network-এর মৌলিক একক, যা input গ্রহণ করে, প্রক্রিয়া করে, এবং output প্রদান করে।

Perceptron

সবচেয়ে সরল Artificial Neuron মডেল, যা মূলত Binary Classification-এর জন্য ব্যবহৃত হয়।

Weight

একটি input-এর গুরুত্ব নির্ধারণকারী সংখ্যাগত মান।

Bias

Weighted Sum-এর সাথে যুক্ত একটি অতিরিক্ত সমন্বয়কারী মান, যা decision boundary স্থানান্তর করতে সাহায্য করে।

Activation Function

Neural Network-এর output-এ Non-linearity যোগ করার function।

Feature

Data-এর একটি পরিমাপযোগ্য বৈশিষ্ট্য, যা model সিদ্ধান্তে ব্যবহার করে।

Feature Engineering

মানুষের domain জ্ঞান ব্যবহার করে raw data থেকে উপযোগী feature তৈরি বা নির্বাচন করার প্রক্রিয়া।

Feature Extraction

Data থেকে উপযোগী বৈশিষ্ট্য বের করে আনার প্রক্রিয়া, যা হাতে বা স্বয়ংক্রিয়ভাবে সম্পন্ন হতে পারে।

Automatic Feature Extraction

Deep Learning model কর্তৃক training-এর মাধ্যমে নিজে থেকেই raw data থেকে feature শেখা।

Representation Learning

Data-কে পরবর্তী কাজের জন্য সবচেয়ে উপযোগীভাবে রূপান্তরিত করে শেখার প্রক্রিয়া।

Structured Data

Row-column কাঠামোতে সংরক্ষিত data, যেমন spreadsheet বা database table।

Unstructured Data

নির্দিষ্ট কাঠামোবিহীন data, যেমন ছবি, টেক্সট, অডিও বা ভিডিও।

Overfitting

যখন একটি model training data অতিরিক্ত মুখস্থ করে ফেলে এবং নতুন data-তে খারাপ performance দেখায়।

Generalization

নতুন, আগে না-দেখা data-তে সঠিকভাবে কাজ করার model-এর ক্ষমতা।

Input Layer

Network-এর প্রথম স্তর, যা raw feature গ্রহণ করে।

Hidden Layer

Input ও Output Layer-এর মধ্যবর্তী স্তর, যেখানে প্রকৃত শেখার প্রক্রিয়া ঘটে।

Output Layer

Network-এর শেষ স্তর, যা চূড়ান্ত prediction তৈরি করে।

Learnable Parameters

Weight ও Bias-এর সমষ্টি, যেগুলোর মান training-এর মাধ্যমে শেখা হয়।

Forward Propagation

Input থেকে Output পর্যন্ত স্তরে স্তরে গণনা এগিয়ে নেওয়ার প্রক্রিয়া, যার ফলাফল হলো prediction।

Loss Function

Prediction ও actual target-এর মধ্যকার পার্থক্য সংখ্যাগতভাবে পরিমাপকারী function।

Backpropagation

Loss থেকে gradient গণনা করে network-এর parameters কীভাবে পরিবর্তন করতে হবে তা নির্ধারণের learning mechanism।

Gradient

একটি parameter পরিবর্তন করলে Loss কতটা পরিবর্তিত হয়, তার পরিমাপ।

Chain Rule

একাধিক স্তরের মধ্য দিয়ে প্রভাব ধারাবাহিকভাবে গণনা করার গাণিতিক নিয়ম, যা Backpropagation-এ ব্যবহৃত হয়।

Optimizer

Gradient ব্যবহার করে Weight ও Bias হালনাগাদকারী algorithm।

Gradient Descent

Loss কমানোর জন্য parameter-কে তার gradient-এর বিপরীত দিকে সরানোর মৌলিক Optimization পদ্ধতি।

Learning Rate

প্রতিটি training ধাপে parameter কতটা পরিবর্তিত হবে তা নির্ধারণকারী মান।

CPU

অল্প সংখ্যক শক্তিশালী core-বিশিষ্ট সাধারণ-উদ্দেশ্য প্রসেসর।

GPU

হাজার হাজার সরল, সমান্তরাল core-বিশিষ্ট প্রসেসর, যা বৃহৎ সমান্তরাল গণনায় দক্ষ।

Parallelism

একই সময়ে একাধিক গণনা সম্পন্ন করার ক্ষমতা।

CUDA

NVIDIA-র তৈরি একটি software platform, যা GPU-তে general-purpose গণনা চালানোর সুযোগ দেয়।

ROCm

AMD-র তৈরি GPU-computing software stack, যা CUDA-র সমতুল্য।

ImageNet

Fei-Fei Li-র নেতৃত্বে তৈরি একটি বৃহৎ, label করা image dataset।

ILSVRC

ImageNet-ভিত্তিক একটি বার্ষিক image classification প্রতিযোগিতা।

AlexNet

Alex Krizhevsky, Ilya Sutskever ও Geoffrey Hinton-এর তৈরি একটি গভীর CNN, যা ২০১২ সালে ILSVRC জয় করে।

CNN (Convolutional Neural Network)

ছবির মতো স্থানিক গঠনসম্পন্ন data-র জন্য উপযোগী একটি Neural Network architecture।

RNN (Recurrent Neural Network)

Sequential data প্রক্রিয়াকরণের জন্য উপযোগী একটি Neural Network architecture, যা Hidden State বজায় রাখে।

LSTM / GRU

RNN-এর উন্নত রূপ, যা দীর্ঘ sequence-এ তথ্য মনে রাখতে বিশেষ gate ব্যবহার করে।

Attention

Model-কে sequence-এর যেকোনো অংশে সরাসরি মনোযোগ দেওয়ার সুযোগ প্রদানকারী পদ্ধতি।

Transformer

সম্পূর্ণভাবে Self-Attention-ভিত্তিক একটি architecture, যা আধুনিক LLM-এর ভিত্তি।

LLM (Large Language Model)

বিপুল পরিমাণ টেক্সট থেকে ভাষার প্যাটার্ন শেখা একটি বিশাল Transformer-ভিত্তিক model।

Token / Tokenization

টেক্সটকে ছোট ছোট একক (token)-এ ভাগ করার প্রক্রিয়া।

Generative AI

নতুন content (টেক্সট, ছবি, অডিও ইত্যাদি) তৈরি করতে সক্ষম AI সিস্টেমের বিস্তৃত ক্ষেত্র।

Quick Revision Sheet

Core Definitions

Artificial Intelligence (AI): কম্পিউটার সিস্টেমকে মানুষের মতো বুদ্ধিবৃত্তিক আচরণ প্রদর্শনের উপযোগী করে তোলার ক্ষেত্র।

Machine Learning (ML): AI-এর একটি উপশাখা, যেখানে সিস্টেম data থেকে patterns শিখে সিদ্ধান্ত নেয়, প্রতিটি নিয়ম হাতে প্রোগ্রাম না করেই।

Deep Learning (DL): ML-এর একটি শাখা, যেখানে বহুস্তরবিশিষ্ট Neural Network ব্যবহার করে data থেকে জটিল patterns শেখানো হয়।

Neuron: Neural Network-এর মৌলিক একক, যা input গ্রহণ করে, প্রক্রিয়া করে, এবং output প্রদান করে।

Perceptron: সবচেয়ে সরল Artificial Neuron মডেল, যা মূলত Binary Classification-এর জন্য ব্যবহৃত হয়।

Weight: একটি input-এর গুরুত্ব নির্ধারণকারী সংখ্যাগত মান।

Bias: Weighted Sum-এর সাথে যুক্ত একটি অতিরিক্ত সমন্বয়কারী মান, যা decision boundary স্থানান্তর করতে সাহায্য করে।

Activation Function: Neural Network-এর output-এ Non-linearity যোগ করার function।

Backpropagation: Loss থেকে gradient গণনা করে network-এর parameters কীভাবে পরিবর্তন করতে হবে তা নির্ধারণের learning mechanism।

Gradient: একটি parameter পরিবর্তন করলে Loss কতটা পরিবর্তিত হয়, তার পরিমাপ।

Optimizer: Gradient ব্যবহার করে Weight ও Bias হালনাগাদকারী algorithm।

CNN (Convolutional Neural Network): ছবির মতো স্থানিক গঠনসম্পন্ন data-র জন্য উপযোগী একটি Neural Network architecture।

RNN (Recurrent Neural Network): Sequential data প্রক্রিয়াকরণের জন্য উপযোগী একটি Neural Network architecture, যা Hidden State বজায় রাখে।

Transformer: সম্পূর্ণভাবে Self-Attention-ভিত্তিক একটি architecture, যা আধুনিক LLM-এর ভিত্তি।

LLM (Large Language Model): বিপুল পরিমাণ টেক্সট থেকে ভাষার প্যাটার্ন শেখা একটি বিশাল Transformer-ভিত্তিক model।

Core Equations

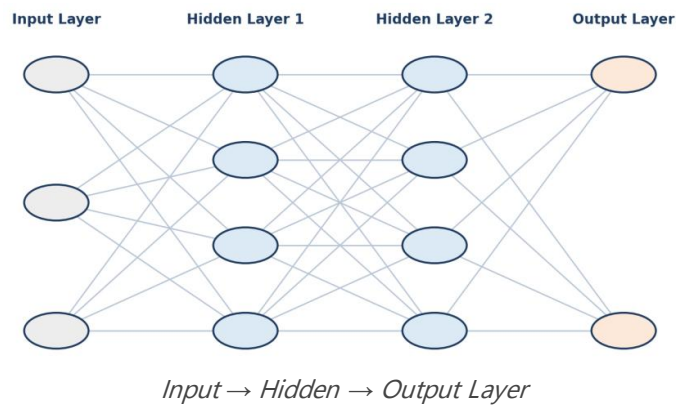
$$z = \sum_i w_i x_i + b$$

$$a = f(z)$$

$$(\partial L) / (\partial w)$$

$$w_{new} = w_{old} - \eta (\partial L) / (\partial w)$$

Core Architecture Diagram

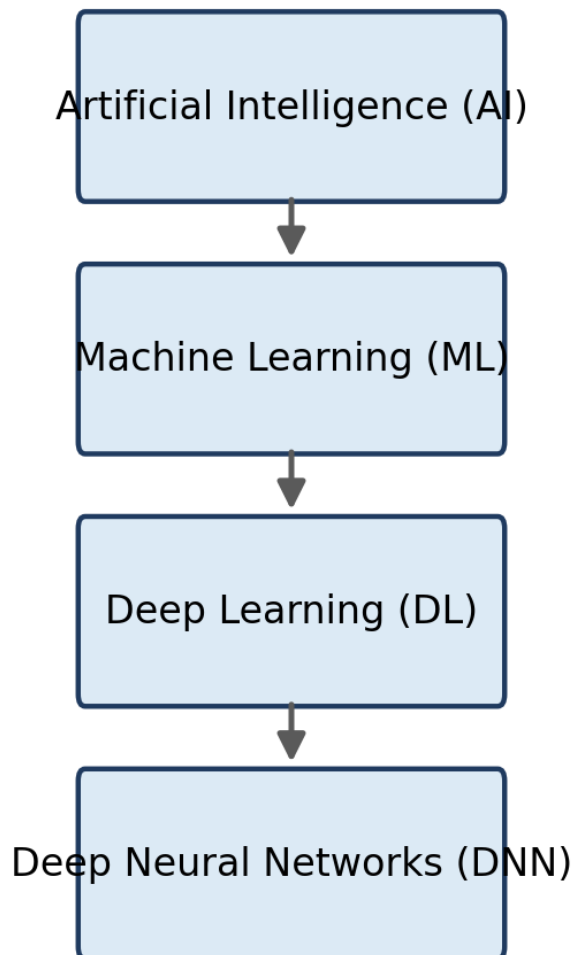


Historical Timeline (সংক্ষেপে)



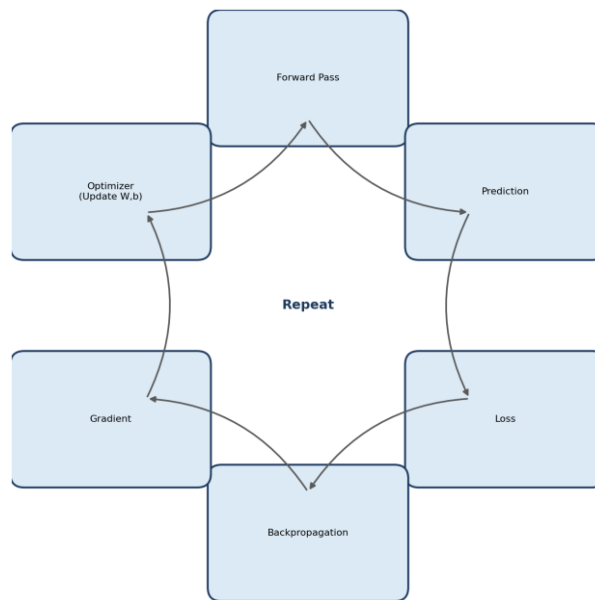
১৯৪৩ → ১৯৫৮ → ১৯৮৬ → AI Winter → ImageNet Era → ২০১২ → ২০১৭

AI → ML → DL সম্পর্ক



Artificial Intelligence → Machine Learning → Deep Learning → Deep Neural Networks

Training Pipeline

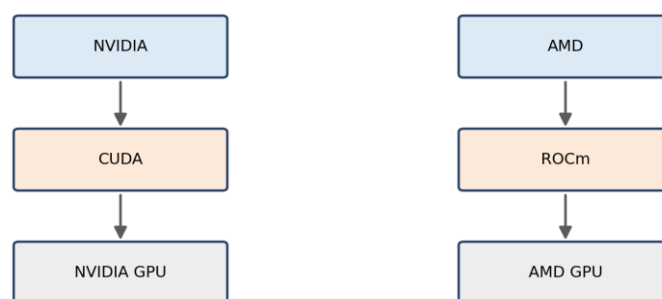


Forward Pass → Prediction → Loss → Backpropagation → Gradient → Optimizer → Update → Repeat

ANN → CNN → RNN → Transformer → LLM

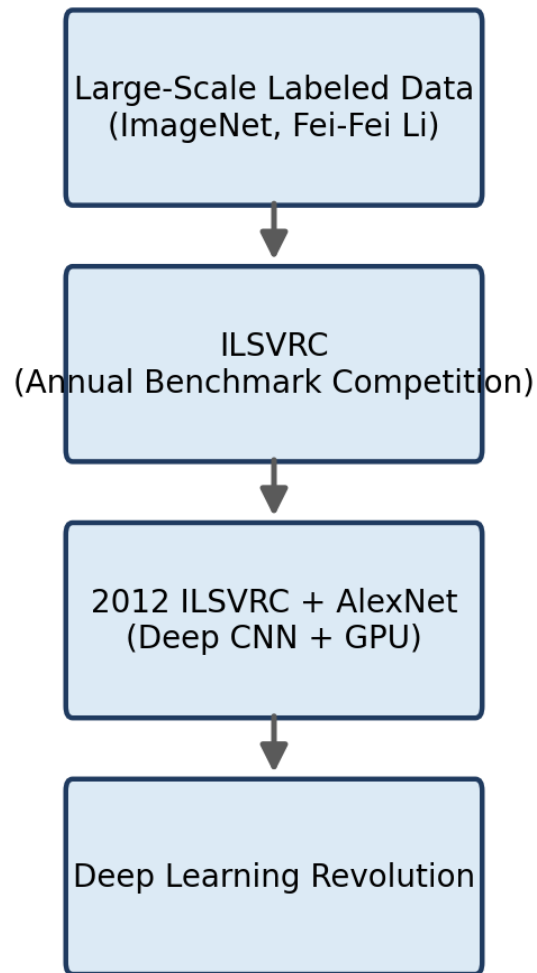
ANN/MLP সাধারণ-উদ্দেশ্য tabular ও সরল সমস্যার জন্য উপযোগী। CNN স্থানিক গঠনসম্পন্ন data (ছবি)-র জন্য বিশেষায়িত। RNN sequential data (টেক্সট, সময়ভিত্তিক data)-র জন্য উপযোগী, যদিও দীর্ঘ sequence-এ সীমাবদ্ধতা রয়েছে। Transformer সম্পূর্ণভাবে Self-Attention-ভিত্তিক, যা parallel processing সম্ভব করে এবং আধুনিক LLM-এর ভিত্তি। LLM হলো Transformer-ভিত্তিক বিশাল model, যা Next-token Prediction-এর মাধ্যমে টেক্সট তৈরি করে।

CPU → GPU → CUDA



NVIDIA → CUDA → GPU এবং AMD → ROCm → GPU

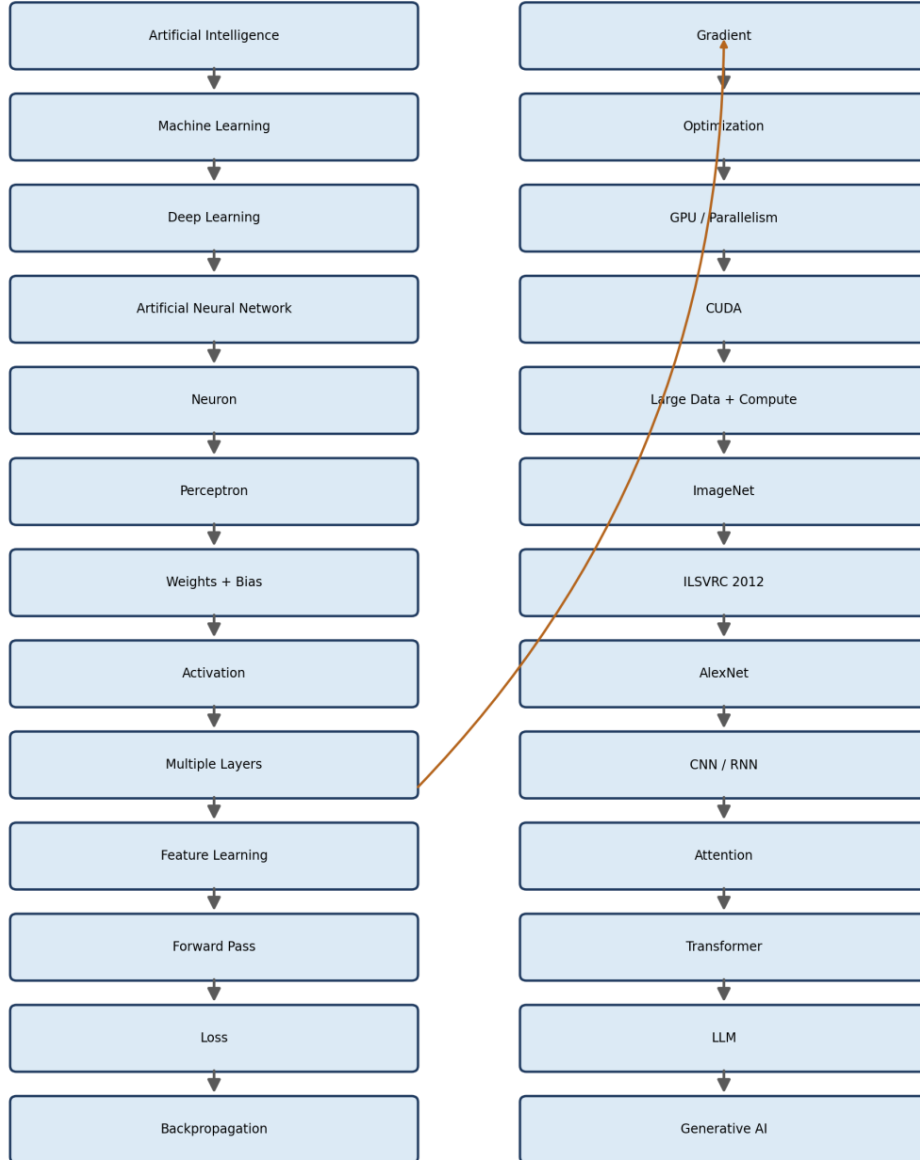
ImageNet → ILSVRC → AlexNet



বৃহৎ Labeled Data → বার্ষিক Benchmark → ২০১২ Deep CNN জয় → Deep Learning বিপ্লব

Final Concept Map

নিচের concept map বইয়ের সম্পূর্ণ ধারণাগত যাত্রাকে একটি একক দৃশ্যে সংক্ষিপ্ত করে — মুদ্রণ (printing) ও দ্রুত পুনরালোচনার জন্য উপযোগী।



Artificial Intelligence থেকে Generative AI পর্যন্ত সম্পূর্ণ ধারণাগত সংযোগ

References

- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. NeurIPS.
- Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature.
- Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. Psychological Review.
- McCulloch, W. S., & Pitts, W. (1943). A Logical Calculus of the Ideas Immanent in Nervous Activity. Bulletin of Mathematical Biophysics.
- Russakovsky, O. et al. (2015). ImageNet Large Scale Visual Recognition Challenge. International Journal of Computer Vision.
- NVIDIA CUDA Documentation — official developer documentation.
- AMD ROCm Documentation — official developer documentation.
- PyTorch Documentation — official framework documentation.